TRƯỜNG ĐẠI HỌC NAM CẦN THƠ

KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO ĐỒ ÁN CƠ SỞ KỸ THUẬT PHẦN MỀM

TÊN ĐỀ TÀI

TRỢ LÝ ẢO SÀNG LỌC TRIỆU CHỨNG

TRẦN HỮU TÀI – 222515

Ngành Kỹ Thuật Phần Mềm

Mã số ngành 7480103

Cần Thơ, tháng năm 2026

TRƯỜNG ĐẠI HỌC NAM CẦN THƠ

KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO ĐỒ ÁN CƠ SỞ KỸ THUẬT PHẦN MỀM

TÊN ĐỀ TÀI

TRỢ LÝ ẢO SÀNG LỌC TRIỆU CHỨNG

Ngành Kỹ Thuật Phần Mềm

Mã số ngành 7480103

Giảng viên hướng dẫn

Giảng viên. Trần Văn Thiện

Sinh viên thực hiện

Trần Hữu Tài – 222515

Cần Thơ, tháng năm 2026

LỜI CẢM ƠN

Với lòng biết ơn sâu sắc và tình cảm chân thành cho phép chúng em gửi lời cảm ơn chân thành nhất tới:

Trường Đại học Nam Cần Thơ, khoa Công Nghệ Thông Tin, cùng các giảng viên đã tận tình chỉ dạy và tạo điều kiện giúp đỡ chúng em trong quá trình học tập, nghiên cứu và hoàn thành đề tài.

Đặc biệt chúng em xin bày tỏ lòng biết ơn sâu sắc đến Thầy Trần Văn Thiện – người hướng dẫn và cũng là người đã luôn tận tình hướng dẫn, chỉ bảo, giúp đỡ và động viên chúng em trong suốt quá trình nghiên cứu và hoàn thành bài đồ án cơ sở này.

Mặc dù, đã cố gắng rất nhiều nhưng bài đồ án không tránh khỏi những thiếu sót, chúng em rất mong nhận được sự thông cảm, chỉ dẫn, giúp đỡ và đóng góp ý kiến của quý thầy cô, các cán bộ quản lý và các bạn đang học cùng chúng em tại Trường Đại học Nam Cần Thơ.

Em xin chân thành cảm ơn!

Cần Thơ, ngày tháng năm 2026

Sinh viên thực hiện

LỜI CAM KẾT

Em xin cam kết đồ án cơ sở kỹ thuật phần mềm này được hoàn thành dựa trên kết quả nghiên cứu của em và kết quả này chưa được sử dụng cho bất kỳ đồ án cơ sở nào khác.

Sinh viên thực hiện

NHẬN XÉT CỦA GIẢNG VIÊN HƯỚNG DẪN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Cần Thơ, ngày .... tháng .....năm 2026

Giảng viên hướng dẫn

NHẬN XÉT CỦA GIẢNG VIÊN PHẢN BIỆN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Cần Thơ, ngày .... tháng .....năm 2026

Giảng viên phản biện

MỤC LỤC

LỜI CẢM ƠN i
LỜI CAM KẾT ii
NHẬN XÉT CỦA GIẢNG VIÊN HƯỚNG DẪN iii
NHẬN XÉT CỦA GIẢNG VIÊN PHẢN BIỆN iv
MỤC LỤC v
DANH MỤC BẢNG vi
DANH MỤC HÌNH vii
DANH MỤC TỪ VIẾT TẮT viii
CHƯƠNG 1 1
GIỚI THIỆU 1
1.1 LÝ DO CHỌN ĐỀ TÀI 1
1.2 MỤC TIÊU NGHIÊN CỨU 2
1.2.1 Mục tiêu tổng quát 2
1.2.2 Mục tiêu cụ thể 2
1.3 ĐỐI TƯỢNG VÀ PHẠM VI NGHIÊN CỨU 3
1.3.1 Đối tượng sử dụng 3
1.3.2 Phạm vi và giới hạn của hệ thống 3
1.4 PHƯƠNG PHÁP NGHIÊN CỨU VÀ CÁC BƯỚC THỰC HIỆN 3
1.4.1 Phương pháp nghiên cứu 3
1.4.2 Trình tự thực thi 4
CHƯƠNG 2 5
CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ ÁP DỤNG 5
2.1 TỔNG QUAN VỀ AI TRONG Y TẾ 5
2.2 MÔ HÌNH NGÔN NGỮ LỚN 6
2.2.1 Khái niệm về mô hình ngôn ngữ lớn 6
2.2.2 Vai trò trong hệ thống 6
2.2.3 Rào cản kỹ thuật 7
2.3 KIẾN TRÚC RAG 7
2.3.1 Khái niệm và Vai trò của RAG 7
2.3.2 Vector Search 8
2.3.3 Keyword Search (BM25) 9

2.3.4 Thuật toán dung hợp Reciprocal Rank Fusion (RRF) 9
2.3.5 Tổng kết luồng kiến trúc Hybrid RAG 10
2.4 KỸ THUẬT PROMPT ENGINEERING 11
2.4.1 Khái niệm 11
2.4.2 Vai trò trong điều hướng hệ thống 11
2.5 CÁC CÔNG CỤ VÀ THƯ VIỆN HỖ TRỢ 12
2.5.1 Retrieval-Augmented Generation 12
2.5.2 Thư viện HuggingFace 12
2.5.3 Thư viện NLP Tiếng Việt 12
2.5.4 Cơ sở dữ liệu Vector 13
2.5.5 Thư viện Rank-BM25 13
2.5.6 Hạ tầng suy luận Groq API 13
2.5.7 Giao diện Streamlit 13
2.5.8 Supabase 14
CHƯƠNG 3 15
PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG 15
3.1 MÔ TẢ HỆ THỐNG 15
3.1.1 Quản lý trạng thái và tiếp nhận truy vấn 15
3.1.2 Truy xuất tri thức lai 15
3.1.3 Bài toán dung hợp 16
3.1.4 Kích hoạt suy luận và ghi đè rủi ro 17
3.2 PHÂN TÍCH YÊU CẦU HỆ THỐNG 17
3.2.1 Yêu cầu chức năng 17
3.2.2 Yêu cầu phi chức năng 18
3.2.3 Mô hình hóa Use Case 19
3.3 KIẾN TRÚC TỔNG THỂ HỆ THỐNG 20
3.4 SƠ ĐỒ KHỐI VÀ LUỒNG XỬ LÝ 22
3.4.1 Luồng xử lý dữ liệu đầu vào 23
3.4.2 Luồng truy vấn và sinh văn bản 25
3.5 THIẾT KẾ CƠ SỞ DỮ LIỆU 28
3.5.1 Cơ sở dữ liệu Vector (ChromaDB) 28
3.5.2 Cơ sở dữ liệu Document Index (BM25) 28
3.5.3 Cơ sở dữ liệu Quan hệ (Supabase) 29

3.5.3.1 Thực thể users (Người dùng) 30
3.5.3.2 Thực thể conversations (Phiên hội thoại) 30
3.5.3.3 Thực thể messages (Chi tiết tin nhắn) 31
3.6 THIẾT KẾ BỘ ĐIỀU PHỐI PROMPT 31
CHƯƠNG 4 34
TRIỂN KHAI VÀ THỬ NGHIỆM HỆ THỐNG 34
4.1 MÔI TRƯỜNG PHÁT TRIỂN VÀ CÀI ĐẶT 34
4.1.1 Môi trường phần cứng 34
4.1.2 Nền tảng phần mềm và thư viện 34
4.2 XÂY DỰNG GIAO DIỆN NGƯỜI DÙNG 34
4.2.1 Bảng điều khiển hội thoại 35
4.2.2 Không gian tương tác chính 35
4.3 KỊCH BẢN THỬ NGHIỆM VÀ ĐÁNH GIÁ 36
4.3.1 Kịch bản 1: Hỏi đáp ngoài phạm vi y tế 36
4.3.1.1 Tiêu điểm kiểm thử 36
4.3.1.2 Dữ liệu đầu vào 36
4.3.1.3 Kết quả xử lý 36
4.3.1.4 Đánh giá kiến trúc 36
4.3.2 Kịch bản 2: Hỏi bệnh thiếu thông tin 37
4.3.2.1 Tiêu điểm kiểm thử 37
4.3.2.2 Dữ liệu đầu vào 37
4.3.2.3 Kết quả xử lý 37
4.3.2.4 Đánh giá kiến trúc 37
4.3.3 Kịch bản 3: Hỏi bệnh đủ thông tin 38
4.3.3.1 Tiêu điểm kiểm thử 38
4.3.3.2 Dữ liệu đầu vào 38
4.3.3.3 Kết quả xử lý 39
4.3.3.4 Đánh giá kiến trúc 39
4.3.4 Kịch bản 4: Tình huống Cấp cứu 39
4.3.4.1 Tiêu điểm kiểm thử 39
4.3.4.2 Dữ liệu đầu vào 40
4.3.4.3 Kết quả xử lý 40
4.3.4.4 Đánh giá kiến trúc 40

4.4 ĐÁNH GIÁ HIỆU NĂNG VÀ ĐỘ CHÍNH XÁC 41
CHƯƠNG 5 42
KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 42
5.1 KẾT QUẢ ĐẠT ĐƯỢC 42
5.2 CÁC HẠN CHẾ CÒN TỒN TẠI 42
5.3 HƯỚNG PHÁT TRIỂN TRONG TƯƠNG LAI 43
TÀI LIỆU THAM KHẢO 45

DANH MỤC BẢNG

Bảng 3.1: So sánh đặc tính truy xuất trong mô hình Hybrid RAG 29

DANH MỤC HÌNH

Hình 2.3.4: Sơ đồ luồng xử lý kiến trúc Hybrid RAG 10
Hình 3.2: Sơ đồ Use Case tổng quát của hệ thống 19
Hình 3.3: Sơ đồ kiến trúc tổng thể hệ thống..... 21
Hình 3.3.1: Sơ đồ hoạt động..... 24
Hình 3.3.2: Sơ đồ tuần tự..... 26
Hình 3.5.3: Sơ đồ thực thể quan hệ 30
Hình 3.6: Sơ đồ khối kiến trúc đóng gói Prompt..... 32
Hình 4.2: Giao diện người dùng 35
Hình 4.3.1: Ảnh hỏi đáp ngoài phạm vi y tế 36
Hình 4.3.2: Ảnh hỏi bệnh thiếu thông tin 37
Hình 4.3.3: Ảnh hỏi bệnh đủ thông tin 39
Hình 4.3.3: Ảnh tình huống cấp cứu 40

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Giải thích
AI	Artificial Intelligence (Trí tuệ nhân tạo)
LLM	Large Language Model (Mô hình Ngôn ngữ Lớn)
RAG	Retrieval-Augmented Generation (Thế hệ tăng cường truy xuất)
NLP	Natural Language Processing (Xử lý ngôn ngữ tự nhiên)
BM25	Best Matching 25 (Thuật toán tìm kiếm từ khóa)
RRF	Reciprocal Rank Fusion (Thuật toán dung hợp xếp hạng)
UI/UX	User Interface / User Experience (Giao diện / Trải nghiệm người dùng)
LPU	Language Processing Unit (Bộ xử lý ngôn ngữ)
CPU	Central Processing Unit (Bộ xử lý trung tâm)
GPU	Graphics Processing Unit (Bộ xử lý đồ họa)
NPU	Neural Processing Unit (Bộ xử lý thần kinh)

CHƯƠNG 1

GIỚI THIỆU

1.1 LÝ DO CHỌN ĐỀ TÀI

Hãy nhìn vào cách một người bình thường xử lý khi họ bị đau ngực hay chóng mặt. Rất hiếm người lập tức chạy đến bệnh viện. Hành vi đầu tiên của họ là mở Google.

Nhưng y tế không giống như tìm chỗ ăn uống. Khi một bệnh nhân ném các triệu chứng rời rạc của mình lên các công cụ tìm kiếm, thứ họ nhận lại là một mớ bòng bong dữ liệu. Vài kết quả từ các trang báo y khoa uy tín bị chôn vùi dưới hàng tá bài viết chuẩn SEO, mẹo vặt dân gian không kiểm chứng, và các quảng cáo thực phẩm chức năng. Hệ lụy thực tế rất tồi tệ: người bệnh tự chẩn đoán sai, tự mua kháng sinh uống vô tội vạ, hoặc rơi vào hội chứng Nocebo tự dọa mình sợ đến mức hoảng loạn vì nghĩ mình mắc bệnh nan y. Các bệnh viện tuyến trên thì vỡ trận vì quá tải, nhưng để mặc bệnh nhân "tự bơi" trên Internet rõ ràng là một lỗ hổng lớn của hệ thống chăm sóc sức khỏe.

Chúng ta từng cố giải bài toán này bằng các Chatbot y tế thế hệ cũ (Rule-based Chatbot). Nhưng chúng thất bại nhanh chóng. Bạn không thể bắt một người đang đau đớn phải bấm từng nút (Button) theo một cây quyết định cứng nhắc dài dằng dặc được trải nghiệm đó quá tệ.

Sự trỗi dậy của các Mô hình Ngôn ngữ Lớn (LLM) như Llama, GPT hay Claude tưởng chừng là chìa khóa vàng. LLM có thể hiểu ngôn ngữ tự nhiên, bắt ngữ cảnh tốt và nói chuyện mượt mà như một bác sĩ thực thụ. Tuy nhiên, mang một mô hình nguyên thủy, chưa qua tinh chỉnh hay kiểm soát ranh giới vào tư vấn sức khỏe lại là một ván cược sinh tử. Bản chất của AI tạo sinh là một cỗ máy dự đoán xác suất từ vựng, và "ảo giác" không phải là lỗi, nó là tính năng của mô hình. Một con LLM có thể cực kỳ tự tin bịa ra một phác đồ điều trị, trích dẫn sai liều lượng thuốc, nhưng lại trình bày với văn phong thuyết phục đến mức người dùng tin sái cổ.

Trong kỹ thuật phần mềm, một cái bug giao diện hay sai số thanh toán có thể tung bản vá trong đêm. Nhưng bug trong y tế phải trả giá bằng tính mạng.

Đứng trước lằn ranh này, giải pháp khả thi nhất không phải là cố dạy cho AI cách làm bác sĩ. Thứ chúng ta cần là một hệ thống đủ linh hoạt để lắng nghe người bệnh, nhưng bị nhốt chặt trong một khung nghiệp vụ khắt khe. Bài toán an toàn và mang lại giá trị thực tiễn cao nhất lúc này là dùng AI để làm nhiệm vụ sàng lọc ban đầu.

Đó là lý do tôi phát triển hệ thống "Trợ lý ảo sàng lọc triệu chứng".

Để triệt tiêu tận gốc căn bệnh ảo giác, tôi tước bỏ hoàn toàn quyền "tự do sáng tác" của LLM bằng kiến trúc truy xuất tăng cường lai Hybrid RAG. Mô hình Llama 3

trong dự án này bị ép buộc chỉ được phép tổng hợp câu trả lời từ một kho y văn đã được làm sạch và kiểm duyệt. Nó hoạt động thuần túy như một nhân viên điều phối y tế tuyến đầu: nghe bệnh nhân kể lể dài dòng, trích xuất chính xác từ khóa triệu chứng, đánh giá mức độ khẩn cấp, phát còi báo động nếu thấy dấu hiệu sinh tồn nguy hiểm, và điều hướng họ đến đúng chuyên khoa. Không chẩn đoán, không kê đơn và chỉ sàng lọc.

1.2 MỤC TIÊU NGHIÊN CỨU

1.2.1 Mục tiêu tổng quát

Thiết kế và triển khai một hệ thống Trợ lý ảo Y tế thấu hiểu tiếng Việt tự nhiên, làm nhiệm vụ phân tích sơ bộ tình trạng người dùng. Đích đến cuối cùng là chứng minh tính khả thi của một kiến trúc AI an toàn: độ trễ thấp, triệt tiêu ảo giác và tuân thủ tuyệt đối quy tắc đạo đức y khoa.

1.2.2 Mục tiêu cụ thể

Dưới lăng kính của người làm phân tích thiết kế hệ thống, để đạt được mục tiêu trên, dự án phải dọn dẹp dứt điểm các điểm nghẽn kỹ thuật sau:

Giải bài toán Data Pipeline: Dữ liệu y tế thô rất lộn xộn. Việc đầu tiên là xây dựng một luồng ETL (Extract, Transform, Load) để thu thập, làm sạch các bộ dữ liệu QA MedQuad và từ điển triệu chứng. Không chặt văn bản một cách máy móc theo số lượng từ, mà phải phân mảnh theo ngữ nghĩa để đảm bảo không làm đứt gãy một bệnh án.

Tối ưu lõi truy xuất: Đây là trái tim của hệ thống. Nếu chỉ dùng Vector Search thuần túy, AI sẽ "mù" trước các từ khóa y khoa. Ví dụ: Vector Search đánh giá "Viêm gan A" và "Viêm gan B" rất giống nhau về mặt ngữ nghĩa, dẫn đến truy xuất sai tài liệu. Để vá lỗ hổng này, hệ thống buộc phải chạy cấu trúc lai Hybrid Search và kết hợp Vector để bắt ngữ nghĩa và BM25 để bắt chính xác từ khóa. Kế tiếp, dùng thuật toán RRF để trộn và xếp hạng kết quả từ cả hai engine trước khi bơm vào làm Context cho LLM.

Xây dựng rào cản an toàn: Prompt Engineering không chỉ là "hỏi khéo", mà là viết code ranh giới cho AI. Sử dụng kỹ thuật định tuyến luồng để nhốt Chatbot vào đúng nghiệp vụ sàng lọc. Đặc biệt, thiết kế một luồng Red Flag Override độc lập khi hệ thống quét thấy các cụm từ đe dọa tính mạng (đau ngực dữ dội, khó thở, nôn ra máu...), nó sẽ lập tức ghi đè mọi quy tắc hội thoại bình thường để bắn tín hiệu cảnh báo cấp cứu.

Quản lý trạng thái vòng đời: Một bệnh nhân không bao giờ khai báo hết bệnh trong một câu. Hệ thống tích hợp Frontend Streamlit và Backend Supabase để quản lý Session, cho phép AI lưu trữ bối cảnh và nhớ các triệu chứng qua nhiều lượt tương tác liên tiếp.

1.3 ĐỐI TƯỢNG VÀ PHẠM VI NGHIÊN CỨU

1.3.1 Đối tượng sử dụng

Người dùng cuối: Những người đang mông lung về tình trạng sức khỏe của mình, cần một công cụ tra cứu nhanh để ra quyết định: ở nhà theo dõi thêm, đi phòng khám, hay phải lao vào viện cấp cứu ngay lập tức.

Các cơ sở y tế và phòng khám: Module này có khả năng nhúng thẳng vào các nền tảng Web/App của bệnh viện. Nó đóng vai trò như một màng lọc tự động, giúp hỏi han và phân luồng bệnh nhân trước khi đẩy thông tin về cho tổng đài viên hoặc bác sĩ trực, giảm tải đáng kể nhân lực vận hành.

Giới hạn tuyệt đối: Hệ thống này không dành cho bác sĩ dùng làm công cụ chẩn đoán lâm sàng. Và nó vĩnh viễn không có giá trị thay thế cho đường dây nóng 115 trong các tình huống nguy kịch.

1.3.2 Phạm vi và giới hạn của hệ thống

Bất kỳ một phần mềm an toàn nào cũng phải xác định rõ giới hạn của nó.

Giới hạn dữ liệu và ngôn ngữ: Chỉ hỗ trợ xử lý ngôn ngữ tiếng Việt. Kho tri thức tập trung giải quyết các nhóm bệnh lý nội/ngoại khoa thông dụng. Các ca bệnh hiếm, chuyên sâu, hoặc liên quan đến nhi khoa/sơ sinh phức tạp nằm ngoài khả năng truy xuất của phiên bản này.

Biên giới chức năng: Hệ thống chỉ thực thi 3 luồng logic: Phân loại mức độ khẩn cấp, cung cấp kiến thức y tế nền, chỉ định chuyên khoa. Mọi nỗ lực của người dùng nhằm ép AI làm toán, viết code, hay hỏi chuyện phiếm đều bị lớp Scope Guard chặn đứng ngay ở tầng ngoài cùng.

Lằn ranh đạo đức Y khoa: Nguyên tắc "3 Không" được hard-code vào nhân hệ thống: Không chẩn đoán định danh bệnh viện, không chỉ định phác đồ điều trị, không kê đơn thuốc dưới mọi hình thức.

1.4 PHƯƠNG PHÁP NGHIÊN CỨU VÀ CÁC BƯỚC THỰC HIỆN

1.4.1 Phương pháp nghiên cứu

Nghiên cứu cơ sở hạ tầng nền: Phân tích đến tận cùng rủi ro của AI tạo sinh để hiểu vì sao phải dùng RAG. Đào sâu vào cách hoạt động của không gian Vector, cách ChromaDB lưu trữ nhúng, và công thức toán học tính điểm TF-IDF của thuật toán BM25.

Thực nghiệm linh hoạt: Trong môi trường RAG, lý thuyết là một chuyện, thực tế chạy ra kết quả lại là chuyện khác. Quá trình phát triển áp dụng tư duy "vừa build, vừa tune". Liên tục chạy A/B Testing để căn chỉnh trọng số Alpha của RRF để xem nên tin

vào Vector Search bao nhiêu %, tin BM25 bao nhiêu %. Tinh chỉnh System Prompt liên tục để cân bằng giữa văn phong đồng cảm và sự cứng rắn của quy định y tế.

Kiểm thử phá vỡ logic: Không đánh giá hệ thống bằng những câu hỏi ngoan ngoãn. Chủ động đóng vai người dùng "độc hại": sử dụng kỹ thuật Prompt Injection để lừa AI vượt rào, cung cấp các triệu chứng lấp lửng đầy mâu thuẫn, hoặc đột ngột thả một từ khóa cấp cứu vào giữa một đoạn chat dài lê thê để xem cơ chế Red Flag có kích hoạt kịp thời hay không.

1.4.2 Trình tự thực thi

Toàn bộ đồ án được kiểm soát và thực thi qua 5 chặng cốt lõi:

Giai đoạn 1 - Nhồi dữ liệu: Viết script cào dữ liệu, dọn dẹp các ký tự rác HTML tags, noise. Thực hiện chia nhỏ văn bản theo cấu trúc ngữ nghĩa, đảm bảo mỗi chunk chứa đủ ngữ cảnh của một triệu chứng độc lập để nhúng hiệu quả.

Giai đoạn 2 - Kiến tạo hạ tầng Hybrid Database: Setup ChromaDB để lưu trữ Vector. Song song đó, khởi tạo và build chỉ mục cho BM25. Viết logic điều phối luồng để hai engine tìm kiếm này hoạt động đồng thời mà không bị ngẽn cổ chai.

Giai đoạn 3 - Tích hợp Core Logic AI: Kết nối Llama 3 thông qua hạ tầng Groq API để giải quyết bài toán độ trễ yếu tố sống còn khi chat. Lập trình thuật toán trộn kết quả RRF. Xây dựng bộ điều hướng hội thoại.

Giai đoạn 4 - Tích hợp Backend và UI: Thiết kế giao diện Chatbot bằng Streamlit sao cho trực quan và tối giản nhất. Hook hệ thống với cơ sở dữ liệu quan hệ Supabase dựa trên PostgreSQL để lưu lại Session ID, quản lý trạng thái phiên làm việc và bảo toàn lịch sử hội thoại của người dùng.

Giai đoạn 5 - QA và tối ưu hiệu năng: Đưa hệ thống vào vòng test cường độ cao. Tập trung vá các lỗi nghiêm trọng, đặc biệt là lỗi kẹt vòng lặp vô hạn infinite loop khi người dùng liên tục từ chối cung cấp thêm thông tin. Rà soát lại tốc độ phản hồi, tinh chỉnh trải nghiệm UI/UX và hoàn thiện.

CHƯƠNG 2

CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ ÁP DỤNG

2.1 TỔNG QUAN VỀ AI TRONG Y TẾ

Nhìn lại những năm qua, nỗ lực số hóa khâu tiếp đón bệnh nhân tại các cơ sở y tế gần như dậm chân tại chỗ. Hầu hết các bệnh viện vẫn đang vắt kiệt nguồn lực cho các hệ thống Rule-based Chatbot. Về bản chất kiến trúc, đây là những cỗ máy trạng thái mang tính xác định. Khái niệm này ám chỉ việc luồng xử lý của hệ thống bị đóng khung hoàn toàn bởi những cây quyết định khổng lồ. Mọi nhánh rẽ, mọi câu hỏi IF-ELSE đều phải được đội ngũ kỹ sư và bác sĩ lâm sàng hard-code thủ công từ trước.

An toàn tuyệt đối? Có thể, nhưng lối thiết kế này vấp phải một giới hạn vật lý không thể vượt qua trong khoa học máy tính là sự bùng nổ tổ hợp.

Bạn không thể nào lập trình sẵn cho hàng vạn cách khác nhau mà một bệnh nhân dùng để mô tả cơn đau ngực. Người thì bảo "đau nhói", người nói "tức ngực", người lại than "nặng lồng ngực như có đá đè". Kết quả của sự cứng nhắc này là trải nghiệm người dùng (UX) bị phá nát. Bệnh nhân những người đang trong cơn đau đớn và hoang mang bị ép phải giao tiếp như một cái máy. Họ phải lướt qua hàng tá menu thả xuống cồng kềnh, cố mò mẫm chọn đúng từ khóa y khoa thì hệ thống mới chịu chạy tiếp.

Sự trỗi dậy của AI tạo sinh đã đập bỏ hoàn toàn lối mòn đó. Khái niệm AI tạo sinh trong xử lý ngôn ngữ tự nhiên đại diện cho một bước nhảy vọt về kiến trúc: chuyển dịch từ hệ thống xác định sang hệ thống xác suất. Thay vì dò tìm chuỗi ký tự khớp xác, phần mềm giờ đây có khả năng nội suy và tiếp nhận ngôn ngữ tự nhiên phi cấu trúc. Người bệnh cứ việc gõ sai chính tả. Cứ việc kể lể dài dòng về việc sáng nay ăn gì trước khi đau bụng. Mạng nê-ron vẫn đủ sức bóc tách chính xác ý định nằm ẩn bên dưới mớ văn bản hỗn độn đó.

Dù vậy, mang AI tạo sinh vào môi trường y tế là một ván cược sinh tử. Đừng nhầm lẫn bài toán này với việc làm một con bot tư vấn bán hàng. Trong một hệ thống E-commerce, thuật toán gợi ý sai một chiếc áo thì rủi ro lớn nhất chỉ là lãng phí một lượt hiển thị. Nhưng trên lâm sàng, mọi thứ đều phải trả giá bằng sinh mạng. Việc AI bỏ lọt một dấu hiệu nhồi máu cơ tim tiềm ẩn chỉ vì người bệnh mô tả sai cách, hoặc thuật toán tự ý nội suy sai liều lượng thuốc từ 50mg thành 500mg, sẽ lập tức dẫn đến thảm họa y khoa và rủi ro pháp lý khổng lồ. AI tạo sinh rất giỏi "đoán", nhưng trong y tế, "đoán" là tối kỵ.

Chính thực tế khốc liệt này đã bẻ lái tư duy thiết kế kiến trúc AI y tế hiện đại. Các SA và kỹ sư phần mềm không còn chạy theo ảo mộng tạo ra một "Bác sĩ ảo" có khả năng chẩn đoán toàn năng nữa. Đó là ranh giới của con người.

Thay vào đó, AI được hạ cấp và định vị lại thành một lớp màng lọc đầu vào. Nó được đặt ở tiền sảnh của hệ thống, đóng vai trò như một bộ phân tích cú pháp siêu việt. Nhiệm vụ duy nhất của khối AI này là gạn lọc các triệu chứng thô từ lời kể lộn xộn, chuẩn hóa chúng thành các trường dữ liệu có cấu trúc, và dán nhãn phân loại mức độ rủi ro (đỏ, vàng, xanh). Từ đó, hệ thống backend mới tiến hành điều hướng dòng người bệnh đến đúng chuyên khoa hoặc kích hoạt báo động cấp cứu. Ranh giới hệ thống được thiết lập cực kỳ sắc bén chỉ hỗ trợ phân luồng, tuyệt đối không chẩn đoán.

2.2 MÔ HÌNH NGÔN NGỮ LỚN

2.2.1 Khái niệm về mô hình ngôn ngữ lớn

Nhiều người, kể cả trong giới công nghệ, vẫn hay lầm tưởng mô hình ngôn ngữ lớn như một thực thể có khả năng thấu hiểu. Dưới lăng kính kiến trúc hệ thống, điều đó hoàn toàn sai. Bản chất của LLM chỉ là một cỗ máy thống kê siêu hạng, được xây dựng trên nền tảng mạng nê-ron học sâu với cấu trúc Transformer.

Thay vì "hiểu" ý nghĩa của một căn bệnh như cách con người làm, cơ chế lõi của LLM là sự chú ý. Khi nhận một chuỗi văn bản đầu vào, hàng tỷ tham số bên trong mạng nê-ron sẽ bị đốt nóng bởi hàng loạt phép nhân ma trận cường độ cao. Mục đích duy nhất của chúng là tính toán phân phối xác suất token nào có khả năng xuất hiện cao nhất ở vị trí tiếp theo. Nó hoàn toàn là một trò chơi điền vào chỗ trống ở quy mô khổng lồ, dựa trên việc nội suy từ hàng terabyte dữ liệu nó từng quét qua.

2.2.2 Vai trò trong hệ thống

Trong thiết kế của hệ thống này, LLM bị giáng cấp. Nó không được phép khoác áo "bác sĩ ảo" để tự suy diễn tri thức y khoa. Tôi thiết lập ranh giới cứng, ép nó lùi về làm một động cơ xử lý ngôn ngữ tự nhiên ở tầng ứng dụng. Nhiệm vụ của nó rất cơ học: hứng mớ câu chữ lộn xộn từ bệnh nhân, bóc tách ý định, suy luận logic theo quy tắc và nhào nặn lại câu trả lời dựa trên lượng ngữ cảnh sạch mà hệ thống RAG vừa mớm cho nó.

Tại sao không bọc API của OpenAI (GPT-4) hay Anthropic cho nhanh? Việc quyết định tích hợp mô hình Llama 3.3 70B Versatile (llama-3.3-70b-versatile) thông qua hạ tầng Groq Cloud API là một chiến lược kiến trúc có chủ ý nhằm giải quyết trọn vẹn bài toán khối lượng tính toán và độ trễ:

- Năng lực suy luận cấp cao: Sàng lọc y tế không phải là trò chuyện phiếm. Để hệ thống tuân thủ nghiêm ngặt các rào cản phủ định, phân tích chính xác các dấu hiệu cấp cứu giấu trong lời kể lộn xộn, và xuất form Markdown chuẩn xác, một mô hình nhỏ rất dễ bị trượt logic. Llama 3.3 với khối lượng 70 tỷ tham số cung cấp sức mạnh nội suy và tuân thủ mệnh lệnh xuất sắc, tiệm cận các mô hình độc quyền hàng đầu nhưng vẫn giữ được sự minh bạch của mã nguồn mở.

- Đập tan nút thắt cổ chai về độ trễ: Chạy một mô hình hạng nặng 70B thông thường sẽ sinh ra độ trễ cực lớn, phá nát trải nghiệm người dùng. Tuy nhiên, bằng việc "ký gửi" luồng suy luận này lên hạ tầng phần cứng của Groq – vận hành trên kiến trúc vi xử lý LPU thiết kế đặc trị để đẩy nhanh tốc độ sinh token hệ thống đã giải quyết được nghịch lý này. Sự kết hợp giữa bộ não 70B và động cơ LPU giúp ép chỉ số Time-To-First-Token xuống mức tiệm cận thời gian thực. Bệnh nhân nhận được luồng phản hồi ngay lập tức, che lấp hoàn toàn cảm giác chờ đợi.

2.2.3 Rào cản kỹ thuật

Kiến trúc dự đoán xác suất tạo nên sức mạnh của LLM, nhưng cũng chính nó đẻ ra một tử huyệt vô phương cứu chữa là hiện tượng ảo giác.

Ảo giác không phải là một cái bug mã nguồn để có thể fix bằng vài dòng lệnh IF-ELSE. Nó là "tính năng" của cơ chế Autoregressive. Hãy hình dung hệ thống nhận được câu hỏi về một ca bệnh hiếm gặp, hoàn toàn nằm ngoài khoảng phân phối của dữ liệu huấn luyện. Lẽ ra hệ thống phải ngắt luồng và trả lời "Tôi không biết". Nhưng không hàm mất mát của Transformer vẫn thúc ép mô hình phải sinh ra chữ để tối ưu điểm số.

Kết quả là nó bắt đầu suy luận mù. Nó tự tin chắp vá các token rác lại với nhau, tạo ra một phác đồ điều trị nghe cực kỳ trôi chảy, văn phong hàn lâm sắc sảo, nhưng sai bét về chuyên môn y khoa. Trong ngành phần mềm thông thường, đoán sai không sao. Trong y tế, đoán sai là chết người. Để ứng dụng LLM vào môi trường không có dung sai cho sai sót này, ta buộc phải tước bỏ vĩnh viễn quyền tự do "nhớ lại" của nó bằng một lớp kiến trúc truy xuất bên ngoài.

2.3 KIẾN TRÚC RAG

2.3.1 Khái niệm và Vai trò của RAG

RAG (Retrieval-Augmented Generation) là một kiến trúc AI kết hợp giữa khả năng truy xuất thông tin (Retrieval) và khả năng tạo sinh văn bản của Mô hình Ngôn ngữ Lớn (LLM). Thay vì để LLM trả lời dựa hoàn toàn vào tham số học được trong quá trình huấn luyện (dễ dẫn đến hiện tượng "ảo giác" - Hallucination), RAG ép buộc mô hình phải tham chiếu đến một cơ sở dữ liệu tri thức bên ngoài đáng tin cậy trước khi sinh ra câu trả lời.

Về mặt lý thuyết Retrieval-Augmented Generation là một khung kiến trúc tiêu chuẩn được thiết kế để khắc phục giới hạn tri thức của các Mô hình ngôn ngữ lớn (LLM). Một quy trình RAG chuẩn mực vận hành qua 3 giai đoạn cốt lõi:

- Retrieval (Truy xuất): Tiếp nhận truy vấn từ người dùng và tìm kiếm các tài liệu có độ liên quan cao nhất từ một cơ sở dữ liệu tri thức ngoại vi.

- Augmentation (Tăng cường): Trích xuất các tài liệu vừa tìm được và định dạng chúng thành một khối "Ngữ cảnh", sau đó nạp chung vào Prompt cùng với truy vấn ban đầu.
- Generation (Sinh văn bản): LLM phân tích khối Prompt đã được tăng cường ngữ cảnh này để sinh ra câu trả lời dựa trên các dữ kiện thực tế, thay vì tự suy diễn.

Chính vì vậy, Retrieval-Augmented Generation (RAG) không phải là một mô hình đơn lẻ, mà là một khung kiến trúc hệ thống. Nó sinh ra để ghép nối hai phần là sức mạnh truy xuất thông tin của các cơ sở dữ liệu truyền thống và năng lực tạo sinh văn bản của các mô hình ngôn ngữ lớn.

Vai trò định tuyến trong hệ thống: Sự tồn tại của RAG trong đồ án này phục vụ một mục đích duy nhất là triệt tiêu ảo giác. Hãy hình dung một LLM nguyên thủy giống như một sinh viên y khoa bước vào phòng thi nhắm mắt làm bài. Cậu ta chỉ có thể dựa vào trí nhớ mông lung từ những gì đã đọc trong quá trình pre-train, rất dễ nhớ sai liều lượng hoặc phác đồ. RAG can thiệp vào luồng này và biến nó thành một kỳ thi mở. Khi luồng truy vấn của người dùng được gửi tới, hệ thống không gọi thẳng API của AI. Thay vào đó, nó rẽ nhánh chui vào kho tri thức nơi lưu trữ các văn bản y khoa đã được thẩm định. Hệ thống trích xuất ra các phác đồ chuẩn xác nhất, nhồi toàn bộ khối dữ liệu này vào Prompt và thiết lập một chỉ thị phần cứng: "Chỉ được đọc ngữ cảnh này để trả lời. Cấm tuyệt đối việc tự suy diễn". Bằng cách này, RAG dùng dữ liệu thực tế làm một cái mỏ neo, xích chặt đầu ra của mạng nê-ron vào sự thật nền.

2.3.2 Vector Search

Làm sao hệ thống biết đoạn văn nào trong CSDL liên quan đến câu hỏi? Chìa khóa là kỹ thuật nhúng. Đây là quá trình toán học hóa văn bản, ánh xạ các chuỗi ký tự phi cấu trúc thành các vector số thực trong một không gian đa chiều. Vector Search hay tìm kiếm ngữ nghĩa, hoạt động dựa trên việc tính toán khoảng cách hình học giữa các vector này, phổ biến nhất là thông qua hàm Cosine Similarity. Theo toán học, các câu có trường nghĩa càng giống nhau thì góc giữa hai vector của chúng càng nhỏ (Cosine tiến về 1).

Rủi ro hệ thống: Sức mạnh của Vector Search nằm ở khả năng bắt được ý định mập mờ. Bệnh nhân gõ "nhức đầu" hay "đau nửa đầu", hệ thống vẫn match được tài liệu chuẩn vì vector của chúng cụm lại với nhau. Tuy nhiên, mang Dense Retrieval quăng vào môi trường y khoa lập tức làm lộ ra một điểm mù chết người: Sự thất bại trước thực thể chuyên ngành. Lấy ví dụ "Viêm gan A" và "Viêm gan B". Trong không gian ngữ nghĩa, hai cụm từ này nằm sát rạt nhau vì bối cảnh của chúng y hệt nhau (gan, virus, vàng da). Nhưng trên lâm sàng, đường lây nhiễm và cách xử trí của chúng là hai thế giới. Tệ hơn là bài toán định lượng. "Paracetamol 500mg" và "Paracetamol 50mg"

chỉ chênh nhau một con số. Vector Search thường xuyên đánh đồng chúng vì khoảng cách ngữ nghĩa quá gần. Nếu chỉ dùng thuật toán này, hệ thống sẽ liên tục lấy râu ông nọ cắm cằm bà kia.

2.3.3 Keyword Search (BM25)

Khái niệm thuật toán: Để vá lỗ hổng của Vector Search, kiến trúc hệ thống buộc phải mở một luồng tìm kiếm song song là tìm kiếm từ khóa thưa. BM25 là thuật toán kinh điển được chọn. Nó vận hành dựa trên công thức TF-IDF tần suất xuất hiện của từ nghịch đảo với độ phổ biến. BM25 phớt lờ hoàn toàn "ý nghĩa" của câu nói. Nó chỉ thống kê khô khan: một từ khóa cụ thể xuất hiện bao nhiêu lần trong tài liệu, và từ đó hiếm đến mức nào trong toàn bộ DB.

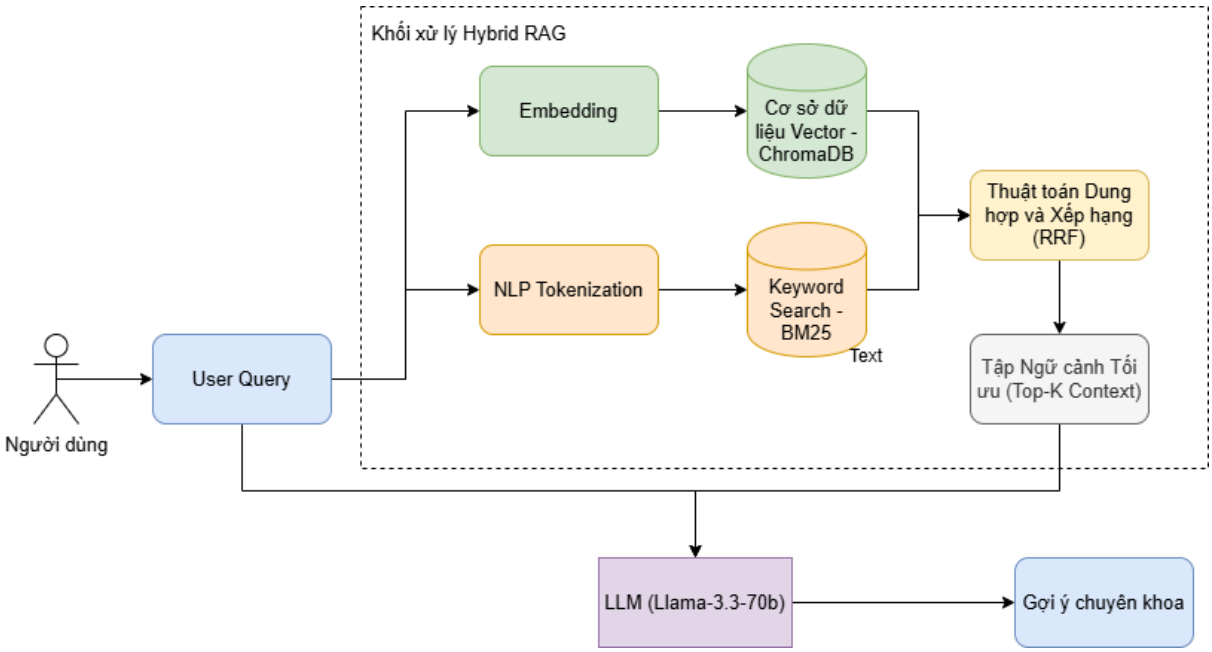
Vai trò và nút thắt Tokenization: BM25 săn lùng sự khớp xác tuyệt đối. Nhờ nó, hệ thống không bao giờ nhầm lẫn giữa 50mg và 500mg. Thế nhưng, triển khai BM25 cho dữ liệu Tiếng Việt là một nỗi đau của người làm hệ thống. Khác với tiếng Anh định danh từ vựng bằng khoảng trắng, Tiếng Việt là ngôn ngữ đơn âm tiết. Một thuật ngữ lâm sàng như "suy thận" mang ý nghĩa trọn vẹn, nhưng thuật toán mặc định sẽ băm nó thành 2 token rời rạc: "suy" và "thận". Hậu quả là BM25 có thể bốc nhầm một tài liệu nói về "suy nghĩ" hay "suy nhược" chỉ vì nó có dính chữ "suy". Để hệ thống tồn tại được, tôi phải thiết kế một Pipeline tiền xử lý (Word Segmentation) chặn ngay ở đầu vào. Toàn bộ câu hỏi của người dùng và dữ liệu kho tri thức đều phải chạy qua thư viện NLP Tiếng Việt để nối cứng các âm tiết có nghĩa lại với nhau bằng dấu gạch dưới (suy_thận, nhồi_máu_cơ_tim). Thao tác này ép engine BM25 phải nhìn nhận thuật ngữ như một thực thể y khoa nguyên khối, chấm dứt tình trạng nhiễu kết quả.

2.3.4 Thuật toán dung hợp Reciprocal Rank Fusion (RRF)

Khái niệm thuật toán: Reciprocal Rank Fusion (RRF) là một thuật toán xếp hạng lại kết quả tìm kiếm. Khác với các phương pháp dung hợp phức tạp, RRF hoàn toàn phớt lờ điểm số tuyệt đối (Score) mà các engine trả về. Nó chỉ quan tâm đến một tham số duy nhất là thứ hạng của tài liệu đó trong từng danh sách. Công thức toán học của nó cực kỳ tối giản: RRF Score = 1 / (k + Rank). Trong đó, k là một hằng số làm mượt (smoothing constant), thường được cấu hình mặc định là 60 nhằm chống lại sự thao túng của các tài liệu rác vô tình lọt top.

Đánh đổi kiến trúc: Tại sao phải dùng RRF? Bởi vì luồng Hybrid Search của chúng ta đang cầm trên tay hai danh sách kết quả không cùng hệ quy chiếu. Điểm Cosine của Vector giới hạn từ -1 đến 1, trong khi điểm TF-IDF của BM25 là con số vô cực không có trần. Việc ép chuẩn hóa (Normalize) hai thang đo này để cộng lại là bất khả thi, vì nó phá hỏng hoàn toàn độ nhạy của thuật toán ban đầu. RRF là lời giải thanh tối ưu nhất. Giả sử một đoạn y văn vừa đủ linh hoạt để hiểu ý định lấp lửng của người dùng (đứng hạng 2 bên bảng Vector), lại vừa chứa chính xác tên loại thuốc họ cần (đứng hạng

4 bên bảng BM25). Thuật toán RRF sẽ cộng dồn trọng số thứ hạng, lập tức đẩy tài liệu này vọt lên vị trí số 1. Luồng xử lý này đảm bảo Context đưa vào cho LLM là một khối thông tin hoàn hảo và không điểm mù ngữ nghĩa, không sai lệch thuật ngữ.



Hình 2.3.4: Sơ đồ luồng xử lý kiến trúc Hybrid RAG

Sơ đồ trên minh họa trực quan luồng xử lý dữ liệu của hệ thống. Theo đó, truy vấn đầu vào được rẽ làm hai nhánh độc lập: nhánh xử lý ngữ nghĩa (truy xuất qua ChromaDB) và nhánh xử lý từ khóa (tiền xử lý hình thái từ tiếng Việt và truy xuất qua BM25). Kết quả từ hai nhánh này được hội tụ tại bộ trộn RRF để tinh lọc ra tập ngữ cảnh tối ưu nhất. Cuối cùng, tập ngữ cảnh này kết hợp cùng truy vấn gốc sẽ được nạp vào LLM làm tri thức nền tảng an toàn để sinh ra câu trả lời định tuyến.

2.3.5 Tổng kết luồng kiến trúc Hybrid RAG

Dựa trên sơ đồ nguyên lý hoạt động, kiến trúc Hybrid RAG giải quyết bài toán thông tin y tế bằng cách tận dụng tối đa thế mạnh của cả hai phương pháp truy xuất. Trong khi luồng Vector Search đảm bảo hệ thống "hiểu" được ý định mơ hồ của bệnh nhân, thì luồng Keyword Search (BM25) lại đóng vai trò là chốt chặn độ chính xác cho các danh từ y khoa đặc thù. Việc sử dụng thuật toán RRF ở giai đoạn cuối giúp dung hợp và tinh lọc ra tập hợp ngữ cảnh chuẩn xác nhất.

Tập ngữ cảnh này chính là nền tảng cốt lõi để giới hạn không gian suy luận của LLM, đảm bảo mọi câu trả lời sinh ra đều được "neo" chặt vào y văn, từ đó loại bỏ hoàn toàn rủi ro ảo giác trong định hướng y tế.

2.4 KỸ THUẬT PROMPT ENGINEERING

2.4.1 Khái niệm

Trên các diễn đàn đại chúng, Prompt Engineering thường bị hiểu sai là những mẹo vặt gõ phím để dụ dỗ Chatbot trả lời hay hơn. Dưới góc độ khoa học máy tính và thiết kế hệ thống, khái niệm này mang một sức nặng hoàn toàn khác.

Trong kiến trúc của một ứng dụng tích hợp LLM, Prompt không phải là văn bản người dùng nhập vào. Prompt là các cấu trúc lệnh đầu vào được mã hóa ở tầng Backend. Vì mạng nê-ron không chạy bằng các khối lệnh if-else hay switch-case tuyến tính, kỹ sư phần mềm buộc phải sử dụng ngôn ngữ tự nhiên như một dạng "ngôn ngữ lập trình bậc cao" mới. Quá trình thiết kế, tinh chỉnh và tối ưu hóa các lệnh này nhằm mục đích duy nhất: thiết lập ranh giới nghiệp vụ, định hình cấu trúc dữ liệu đầu ra và lập trình hành vi cho LLM như một cỗ máy trạng thái.

2.4.2 Vai trò trong điều hướng hệ thống

Đưa một mô hình ngôn ngữ thô vào môi trường y tế mà chỉ dùng các prompt tĩnh chẳng khác nào thả một quả bom nổ chậm. Hệ thống trong đồ án này không hoạt động như vậy.

Tôi thiết kế một kiến trúc System Prompt phân tầng, được tiêm (inject) trực tiếp vào payload của API trước khi gửi đến Llama 3. Mục tiêu là xây dựng một cái lồng vô hình, "nhốt" mô hình này vào một ranh giới an toàn tuyệt đối thông qua ba chốt chặn kỹ thuật:

Trấn áp tính "hầu hạ" (Persona Shaping): Các mô hình AI hiện đại đều trải qua quá trình RLHF học tăng cường từ phản hồi của con người, khiến chúng mang một căn bệnh rất khó chịu cho dân làm hệ thống là chúng quá lịch sự và nói quá nhiều. Chúng thích đóng vai một chuyên gia tâm lý lan man. Prompt ở tầng này thực hiện thao tác xóa bỏ nhân cách đó. Hệ thống ép Llama 3 đóng vai một nhân viên điều phối lạnh lùng, vô cảm và chính xác. Lệnh được hard-code trực tiếp không chào hỏi, không an ủi và bỏ qua mọi từ ngữ thừa thãi. Nhiệm vụ duy nhất của mi là trích xuất dữ kiện bệnh lý.

Rào cản phủ định: Đây là lớp tường lửa của ứng dụng. Người dùng cuối rất khó đoán, họ có thể dùng kỹ thuật Prompt Injection để ép hệ thống tự viết đơn thuốc hoặc chuyển sang hỏi code Python. Thay vì chỉ bảo AI "hãy làm gì", Negative Prompting chỉ định rõ ràng những vùng cấm tuyệt đối. Các chỉ thị phủ định cứng được đính kèm vào mọi lượt chat: "Tuyệt đối không chẩn đoán định danh căn bệnh. Không bao giờ cung cấp tên thuốc hoặc liều lượng. Ngắt luồng hội thoại và trả về mã lỗi nếu phát hiện câu hỏi liên quan đến chính trị, toán học hoặc lập trình". Lớp rào cản này mạnh đến mức nó sẽ ghi đè lên trọng số khao khát trả lời tự nhiên của mô hình.

Định tuyến động và lấp đầy tham số: Đây là phần phức tạp nhất trong kiến trúc điều khiển. Một AI tồi là một AI vội vã kết luận. Nếu bệnh nhân nhập "tôi bị đau bụng", mô hình không được phép đưa ra đánh giá phân luồng ngay lập tức. Prompt được thiết kế để ép AI rơi vào vòng lặp trạng thái thu thập thông tin. Nó tự động quét câu nói của bệnh nhân xem đã đủ các biến số tối thiểu chưa. Đau ở vùng nào? Đau âm ỉ hay quặn thắt? Có đi kèm nôn mửa hay sốt không? Nếu các tham số này vẫn đang mang giá trị null, Prompt ép AI tạo ra câu hỏi truy vấn ngược lại người dùng. Cỗ máy trạng thái chỉ chuyển sang bước đánh giá mức độ khẩn cấp khi và chỉ khi bộ Context đã được gom đầy đủ. Cơ chế này ngăn chặn hoàn toàn việc chẩn đoán mò dựa trên dữ liệu thiếu hụt.

2.5 CÁC CÔNG CỤ VÀ THƯ VIỆN HỖ TRỢ

Bản vẽ kiến trúc cần những vật liệu cụ thể để thi công. Để hiện thực hóa luồng dữ liệu của hệ thống, đồ án sử dụng ngôn ngữ lõi là Python và cấu trúc nên một hệ sinh thái các công cụ chuyên sâu như sau:

2.5.1 Retrieval-Augmented Generation

Khái niệm: Đây là các bộ khung phần mềm chuyên dụng dùng để phát triển các ứng dụng dựa trên mô hình ngôn ngữ lớn. Chúng cung cấp các lớp trừu tượng để chuẩn hóa việc kết nối mô hình ngôn ngữ với các nguồn dữ liệu bên ngoài.

Vai trò trong dự án: Hoạt động như một "tổng công trình sư" điều phối toàn bộ luồng RAG. Thay vì phải tự code thủ công hàng ngàn dòng lệnh để kết nối cơ sở dữ liệu với AI, hệ thống dùng framework này để đóng gói quy trình: băm nhỏ tài liệu, gọi bộ truy xuất, và tự động bơm ngữ cảnh vào Prompt trước khi gửi truy vấn đến LLM. Nó giúp mã nguồn module hóa và dễ bảo trì.

2.5.2 Thư viện HuggingFace

Khái niệm: Một thư viện mã nguồn mở bằng Python, cung cấp các mô hình học sâu đã được huấn luyện sẵn để thực hiện quá trình nhúng văn bản thành các ma trận vector số học.

Vai trò trong dự án: Chịu trách nhiệm khởi tạo vector cho toàn bộ kho y văn và câu hỏi của bệnh nhân. Đồ án dùng thư viện này để tải các mô hình nhúng tiếng Việt về chạy trực tiếp trên máy chủ cục bộ. Vai trò cốt lõi của nó là giữ an toàn dữ liệu: hệ thống hoàn toàn không phải gọi API ra môi trường mạng bên ngoài để lấy vector, triệt tiêu rủi ro rò rỉ bệnh án.

2.5.3 Thư viện NLP Tiếng Việt

Khái niệm: Là các bộ công cụ xử lý ngôn ngữ tự nhiên được thiết kế đặc trị cho bộ quy tắc ngữ pháp Tiếng Việt, nổi bật với tính năng Word Segmentation và POS Tagging.

Vai trò trong dự án: Đóng vai trò làm màng lọc tiền xử lý. Do Tiếng Việt là ngôn ngữ đơn âm tiết, các thư viện này can thiệp vào chuỗi văn bản đầu vào để nối cứng các âm tiết có nghĩa lại với nhau (ví dụ: đau_dạ_dày). Nhiệm vụ của nó là chuẩn bị một bộ dữ liệu nguyên khối, giúp thuật toán tìm kiếm từ khóa bên dưới hiểu đúng thuật ngữ y khoa mà không bị băm nát ngữ nghĩa.

2.5.4 Cơ sở dữ liệu Vector

Khái niệm: Một cơ sở dữ liệu Vector mã nguồn mở. Nó được thiết kế chuyên biệt để lưu trữ các bản ghi dưới dạng Embeddings đa chiều và thực thi các phép toán đo lường khoảng cách với tốc độ cực cao.

Vai trò trong dự án: Làm hạ tầng lưu trữ chính cho luồng tìm kiếm ngữ nghĩa. ChromaDB được cấu hình để chạy trực tiếp trên bộ nhớ RAM tại local thay vì dùng dịch vụ Cloud. Vai trò của nó là quét và tính toán điểm Cosine Similarity ngay tức thì, triệt tiêu toàn bộ độ trễ mạng.

2.5.5 Thư viện Rank-BM25

Khái niệm: Một bản triển khai bằng ngôn ngữ Python của tập hợp các thuật toán tìm kiếm thưa. Cốt lõi của nó là thuật toán BM25, vận hành dựa trên cơ chế đánh giá điểm số bằng hàm TF-IDF.

Vai trò trong dự án: Gánh vác luồng tìm kiếm từ khóa độc lập. Rank-BM25 chạy song song với ChromaDB, làm nhiệm vụ "săn lùng" các thuật ngữ lâm sàng đặc thù hoặc các chỉ số định lượng liều lượng thuốc (như 50mg, 500mg) với độ khớp xác tuyệt đối. Nó là mảnh ghép bù đắp cho điểm mù ngữ nghĩa của Vector Search.

2.5.6 Hạ tầng suy luận Groq API

Khái niệm: Nền tảng cung cấp dịch vụ suy luận mô hình AI đám mây. Khác với các cụm server truyền thống chạy GPU, Groq sử dụng vi xử lý LPU kiến trúc SRAM được thiết kế đặc trị để đẩy nhanh tốc độ sinh token.

Vai trò trong dự án: Giải quyết nút thắt cổ chai về độ trễ. Ứng dụng y tế yêu cầu phản hồi ngay lập tức. Vai trò của Groq API là cung cấp năng lượng tính toán để chạy lõi mô hình Llama 3 8B, đẩy tốc độ sinh chữ lên hàng trăm token/giây, tạo ra thời gian chờ cực thấp.

2.5.7 Giao diện Streamlit

Khái niệm nền tảng: Streamlit không giống các framework Frontend truyền thống như React hay Vue. Nó là một thư viện mã nguồn mở bằng Python được thiết kế đặc trị cho giới kỹ sư dữ liệu và AI. Công cụ này cho phép dựng các ứng dụng web trực quan chỉ bằng vài chục dòng code Python thuần mà không cần đụng đến HTML, CSS hay JavaScript.

Vai trò và nút thắt kỹ thuật: Trong kiến trúc hệ thống, Streamlit gánh vác toàn bộ lớp hiển thị. Nó chịu trách nhiệm render giao diện Chatbot, tiếp nhận triệu chứng và trả về kết quả cho bệnh nhân. Tuy nhiên, mang Streamlit vào làm ứng dụng Chatbot đụng ngay một giới hạn vật lý chí mạng là bản chất của giao thức này là phi trạng thái. Nghĩa là, mỗi khi người bệnh gõ một câu mới và ấn Enter, toàn bộ mã nguồn sẽ tự động load lại từ đầu. Nếu để chạy tự nhiên, AI sẽ bị "mất trí nhớ" ngay lập tức, quên sạch mọi triệu chứng mà bệnh nhân vừa khai báo ở lượt chat trước.

2.5.8 Supabase

Khái niệm nền tảng: Supabase là một nền tảng Backend-as-a-Service mã nguồn mở. Khác với các giải pháp NoSQL, Supabase vận hành dựa trên lõi là hệ quản trị cơ sở dữ liệu quan hệ PostgreSQL cực kỳ mạnh mẽ, đi kèm sẵn các tính năng xác thực người dùng và API thời gian thực.

Vai trò kiến trúc: Sự xuất hiện của Supabase trong hệ thống chính là liều thuốc chữa căn bệnh "mất trí nhớ" của lớp Frontend. Nó được móc nối trực tiếp vào luồng dữ liệu để làm một vùng nhớ bền vững. Khi bệnh nhân bắt đầu phiên khám, Supabase đứng ở tầng dưới tự động cấp phát một Session ID độc lập. Mọi câu hỏi và câu trả lời đều được cập nhật và lưu vào cơ sở dữ liệu quan hệ. Nhờ luồng quản lý phiên làm việc này, Llama 3 mới có thể truy xuất ngược lại bối cảnh cũ, xâu chuỗi hàng tá câu nói rời rạc của người bệnh thành một bệnh án hoàn chỉnh xuyên suốt cuộc tư vấn đa lượt.

CHƯƠNG 3

PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1 MÔ TẢ HỆ THỐNG

Nhìn từ góc độ kiến trúc, Hệ thống Trợ lý Sàng lọc Y tế không đơn thuần là một ứng dụng Client-Server bọc API của một mô hình ngôn ngữ lớn. Đưa một mô hình tạo sinh thuần túy vào môi trường y tế là một rủi ro không thể chấp nhận. Bản chất của LLM là dự đoán xác suất, và nó sẵn sàng "bịa" ra một phác đồ điều trị nếu thiếu dữ kiện. Do đó, ranh giới hệ thống buộc phải được thiết kế xoay quanh một đường ống kìm hãm là dùng sự khô khan, chính xác của truy xuất thông tin để xích chặt giới hạn tạo sinh của AI.

Toàn bộ đường ống xử lý lõi vận hành theo một luồng kỹ thuật đơn hướng và nghiêm ngặt: Truy vấn đầu vào (User Query) -> Tìm kiếm lai (Vector + BM25) -> Dung hợp và xếp hạng (RRF) -> Suy luận có kiểm soát (Prompt + LLM).

Đi sâu vào luồng thực thi, vòng đời của một truy vấn sẽ đi qua 4 màng lọc kiến trúc sau:

3.1.1 Quản lý trạng thái và tiếp nhận truy vấn

Ở tầng giao diện, bệnh nhân tương tác thông qua Streamlit. Vấn đề cốt lõi của các framework Web dạng này là bản chất phi trạng thái. Trong khám bệnh lâm sàng, thông tin hiếm khi đến từ một câu nói duy nhất; bệnh nhân thường rỉ rả từng triệu chứng qua nhiều lượt thoại. "Sáng nay tôi buồn nôn", rồi ba lượt chat sau mới bổ sung "có kèm theo đau nửa đầu". Vì vậy, ngay khi Frontend nhận truy vấn, hệ thống Backend không ném nó đi xử lý ngay. Nó phải móc nối với cơ sở dữ liệu quan hệ Supabase để định danh và bốc toàn bộ lịch sử của phiên làm việc đó lên. Không có bước phục hồi bối cảnh này, AI sẽ mắc chứng mất trí nhớ ngắn hạn và đánh giá sai hoàn toàn diễn tiến bệnh.

3.1.2 Truy xuất tri thức lai

Đây là chốt chặn sinh tử để chống ảo giác y khoa. Thay vì để AI tự do lục lọi mớ kiến thức nội tại rỗng tuếch của nó, truy vấn bị ép đi qua hai luồng tìm kiếm độc lập trên kho y văn cục bộ:

- Dense Retrieval (ChromaDB): Luồng này chạy thuật toán Vector Search dựa trên điểm Cosine Similarity. Ưu điểm của Vector là khả năng bắt ngữ nghĩa lấp lửng. Nó thừa sức hiểu "đau bao tử" và "viêm loét dạ dày" chung một cụm ý định. Nhưng không gian Vector lại cực kỳ ngốc trước các chỉ số định lượng hoặc biệt dược chính xác (khoảng cách ngữ nghĩa giữa thuốc 50mg và 500mg là rất nhỏ, dễ dẫn đến truy xuất nhầm).

- Sparse Retrieval (rank_bm25): Để trám vào điểm mù của Vector, luồng tìm kiếm từ khóa thưa BM25 nhảy vào. Nó hoạt động dựa trên điểm TF-IDF, bỏ qua hoàn toàn "ý nghĩa" của câu nói để tập trung bắt chính xác độ khớp (exact match) của các thuật ngữ chuyên ngành và con số định lượng. Mỗi engine sẽ tự chạy và ném về top 5 tài liệu tiềm năng nhất theo hệ quy chiếu của riêng nó.

Quy trình thực thi luồng Hybrid RAG trong hệ thống. Khác với các mô hình RAG tiêu chuẩn chỉ sử dụng một Vector Database, hệ thống này triển khai luồng Hybrid RAG hai pha (Two-phase Hybrid RAG) để đảm bảo không bỏ lọt từ khóa y khoa. Quá trình này được thiết kế như sau:

- Bước 1 - Trích xuất đa luồng (Dual Retrieval): Khi nhận truy vấn đã qua tiền xử lý, hệ thống đồng thời gọi hai Engine. Engine thứ nhất càn quét ChromaDB (Dense Retrieval) để gom top tài liệu có độ tương đồng ngữ nghĩa. Engine thứ hai chạy thuật toán BM25 (Sparse Retrieval) để bắt chính xác các danh từ biệt dược hoặc mã bệnh.
- Bước 2 - Dung hợp ngữ cảnh (Context Fusion): Hai danh sách kết quả độc lập được đẩy vào bộ trộn. Thuật toán Reciprocal Rank Fusion (RRF) sẽ tính toán và xếp hạng chéo lại toàn bộ tài liệu dựa trên vị trí của chúng ở cả hai luồng, loại bỏ các tài liệu nhiễu.
- Bước 3 - Tăng cường Prompt (Prompt Augmentation): Tập ngữ cảnh tối ưu nhất (Top-K Context) sau khi dung hợp được hệ thống đóng gói, nhồi chung với câu hỏi của người dùng và các rào cản an toàn y tế vào một System Prompt cấu trúc.
- Bước 4 - Sinh văn bản có kiểm soát (Grounded Generation): Khối Payload hoàn chỉnh được gửi đến Llama-3.3. Mô hình bị ép buộc chỉ được phép nội suy câu trả lời dựa trên lượng ngữ cảnh đã cung cấp, qua đó hoàn thành quy trình RAG và triệt tiêu hoàn toàn rủi ro ảo giác tự sáng tác.

3.1.3 Bài toán dung hợp

Hệ thống cầm trên tay hai danh sách kết quả, nhưng điểm số của chúng không cùng "ngôn ngữ". Cosine Similarity đo góc (chạy từ -1 đến 1), trong khi điểm TF-IDF không có mức trần. Việc chuẩn hóa và cộng hai con số này lại là vô nghĩa về mặt toán học. Giải pháp được thiết kế ở đây là RRF. Thuật toán này phớt lờ hoàn toàn điểm số tuyệt đối, chỉ làm việc với thứ hạng. Nó tiến hành gộp danh sách, đánh sập những tài liệu nhiễu vô tình lọt top ở một bên, và kéo những tài liệu xuất hiện ổn định ở cả hai luồng lên đầu. Kết quả đầu ra là một khối top 3 tài liệu y khoa đặc nhất, sạch nhất, sẵn sàng làm bệ phóng cho LLM.

3.1.4 Kích hoạt suy luận và ghi đè rủi ro

Top 3 tài liệu trên không được xuất thẳng ra màn hình. Chúng được đóng gói thành một Payload khổng lồ, nhồi chung với lịch sử chat và truy vấn hiện tại, ép vào một System Prompt có cấu trúc đóng. Khối dữ liệu này được bắn qua hạ tầng Groq API để gọi mô hình Llama 3. Việc chọn Groq thay vì các nền tảng khác giải quyết trực tiếp bài toán Time-To-First-Token, độ trễ thấp là yếu tố bắt buộc để giữ chân người dùng trong ứng dụng chat.

Tại bước này, Llama 3 bị tước quyền chẩn đoán. Nó chỉ hoạt động như một cỗ máy tổng hợp văn bản bị trói buộc bởi các quy tắc nghiệp vụ cứng: Đầu tiên, nó phải quét qua màng lọc dấu hiệu cấp cứu. Nếu phát hiện cụm từ nguy hiểm đe dọa sinh tồn (như đau ngực dữ dội, nôn ra máu), hệ thống lập tức ghi đè luồng hội thoại bình thường để bắn cảnh báo đỏ. Nếu an toàn, nó mới bắt đầu dán nhãn phân loại mức độ khẩn cấp và kết xuất lời khuyên định hướng chuyên khoa. Bất cứ câu trả lời nào có dấu hiệu vượt rào như tự ý kê đơn thuốc đều bị Prompt kỹ thuật chặn đứng ngay trước khi stream về phía client.

3.2 PHÂN TÍCH YÊU CẦU HỆ THỐNG

Phân tích yêu cầu trong một dự án có yếu tố y tế không phải là việc lên danh sách tính năng cho có. Quá trình này đóng vai trò như một bản hợp đồng kiến trúc, bóc tách hệ thống thành hai khối ranh giới rõ rệt là những gì cỗ máy này bắt buộc phải thực thi về yêu cầu chức năng và những giới hạn kỹ thuật nó không bao giờ được phép vi phạm về mặt yêu cầu phi chức năng.

3.2.1 Yêu cầu chức năng

Hệ thống không vận hành nguyên khối mà được chia tách thành các phân hệ độc lập để dễ kiểm soát bối cảnh hội thoại và định tuyến luồng dữ liệu. Cụ thể:

Phân hệ định danh và quản lý phiên:

- Bài toán ở đây là phải xử lý mượt mà cả hai tệp người dùng: khách vãng lai muốn hỏi nhanh rồi thoát, và thành viên định danh cần lưu trữ hồ sơ. Tuy nhiên, yêu cầu kỹ thuật sống còn nằm ở cơ chế phân mảnh phiên làm việc. Mỗi lần người dùng mở chat, hệ thống phải cấp phát một Session ID riêng biệt trên cơ sở dữ liệu. Mọi bối cảnh bệnh lý, triệu chứng của phiên nào phải đóng gói kín trong phiên đó, tuyệt đối không được rò rỉ hay lẫn lộn context sang các lần tư vấn khác.

Phân hệ xử lý ngôn ngữ và trạng thái hội thoại:

- Không ép người bệnh phải lướt menu hay chọn dropdown. Hệ thống phải hứng được đầu vào là văn bản thô, bất chấp việc gõ sai chính tả hay thiếu dấu. Đặc

biệt, cỗ máy trạng thái phải được lập trình để xử lý cơ chế Slot Filling. Nghĩa là, nếu người dùng thả một câu lấp lửng kiểu "tôi đau bụng quá", hệ thống không được phép đoán mò mà phải ngắt luồng, chủ động đặt câu hỏi vặn vẹo ngược lại (đau bao lâu rồi? quặn thắt hay âm ỉ?) cho đến khi lấp đầy các tham số y tế cơ bản.

Phân hệ suy luận và ngắt luồng cấp cứu:

- Đây là bộ não của ứng dụng. Yêu cầu đầu tiên là luồng RAG phải luôn chạy trước để bốc dữ liệu nền làm bệ đỡ. Kế tiếp, hệ thống phải duy trì một cơ chế Rẽ nhánh ưu tiên. Bất cứ lúc nào engine phát hiện các cụm từ đe dọa sinh tồn (ví dụ: "tức ngực thở không được", "nôn ra máu"), nó phải bypass toàn bộ logic tư vấn thông thường, lập tức giật còi báo động và ép xuất thông báo gọi cấp cứu. Với các ca nhẹ hơn, hệ thống mới được phép chạy Triage để phân loại và chỉ định chuyên khoa.

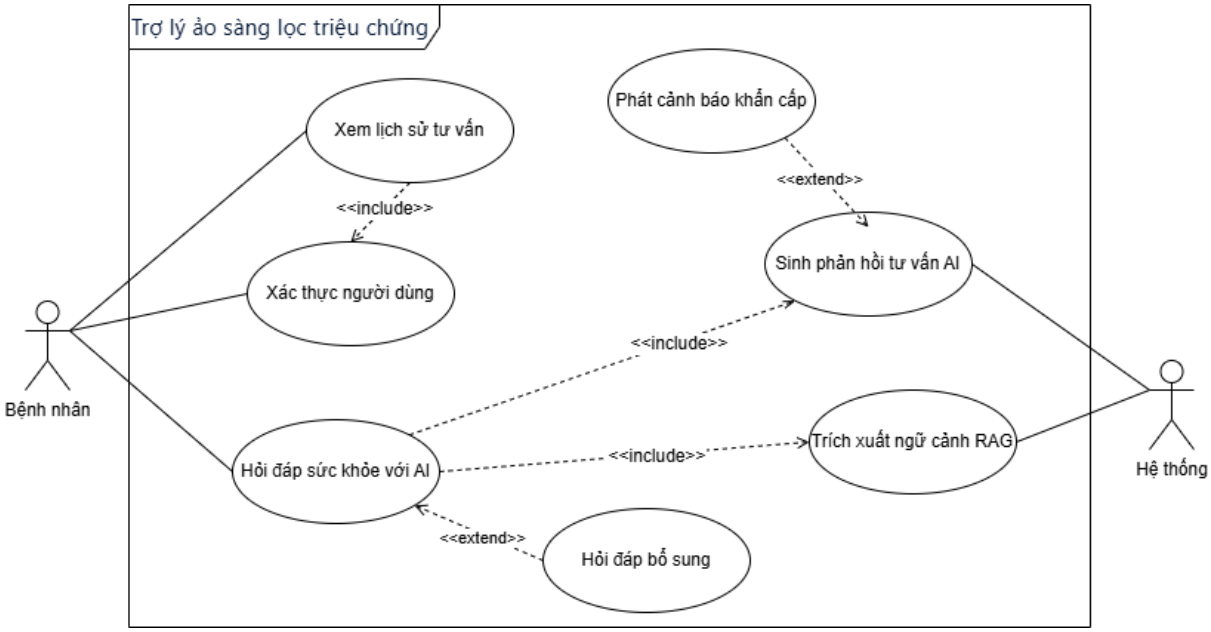
3.2.2 Yêu cầu phi chức năng

Trong môi trường phần mềm y khoa, đôi khi yêu cầu phi chức năng mới là thứ quyết định dự án có sống sót được hay không. Hệ thống bị ép vào các SLA cực kỳ khắt khe:

- Độ trễ: Trải nghiệm Chatbot là phải phản hồi ngay. Thời gian từ lúc người dùng ấn Enter đến khi ký tự đầu tiên nhảy lên màn hình không được vượt ngưỡng 3 giây. Tổng thời gian chạy cào Vector, trộn RRF và sinh chuỗi văn bản hoàn chỉnh chỉ được dao động từ 5-8 giây. Chậm hơn mức này, luồng tương tác bị gãy.
- Kháng ảo giác tuyệt đối: Đây không phải là tính năng "nice-to-have", đây là lằn ranh đạo đức y khoa. LLM bị tước quyền tự do sáng tác. Mọi kết xuất bắt buộc phải grounding trên cơ sở dữ liệu y văn nội bộ. Hệ thống được hard-code khóa chặt hai nghiệp vụ: Cấm chẩn đoán định danh căn bệnh và cấm tư vấn liều lượng biệt dược dưới mọi hình thức.
- Bảo mật và đóng gói dữ liệu: Dữ liệu y tế là mục tiêu nhạy cảm. Toàn bộ mật khẩu lưu trữ phải được băm qua thuật toán Bcrypt. Khối lượng payload bao gồm câu hỏi và bối cảnh bệnh lý không được phép đẩy qua các API LLM công cộng không có cam kết bảo vệ dữ liệu nhằm chặn đứng rủi ro rò rỉ hồ sơ cá nhân.
- Khả năng sử dụng trong trạng thái hoảng loạn: Giao diện (UI) phải bị ép xuống mức tối giản nhất. Một người đang đau đớn hay hoảng sợ vì bệnh tật không có tâm trí đâu mà đọc hướng dẫn sử dụng hay tìm kiếm nút bấm. Chỉ cần một ô nhập liệu, một nút gửi, chấm hết.

3.2.3 Mô hình hóa Use Case

Dựa trên khối yêu cầu, bản vẽ Use Case được thiết lập để quy hoạch lại ranh giới tương tác giữa người dùng cuối và các tiến trình chạy ngầm.



Hình 3.2: Sơ đồ Use Case tổng quát của hệ thống

Hai nhóm tác nhân được định vị ở hai thái cực. Bệnh nhân (Primary Actor) đóng vai trò là điểm kích hoạt, ném vào hệ thống những truy vấn thô, lộn xộn. Ở đầu bên kia, Hệ thống Backend gánh vác toàn bộ sức nặng của bài toán. Nó không có giao diện, vận hành như một cỗ máy tính toán vô hình để nhai nuốt dữ liệu, chạy các thuật toán NLP và cày xới kho y văn khổng lồ. Sự phân vai này được trói chặt bởi hai cơ chế định tuyến logic:

Cơ chế ép buộc (<<include>>): Các rào cản không thể phá vỡ đây là những thỏa thuận phần cứng mang tính sống còn. Nhìn vào cụm chức năng Hỏi đáp sức khỏe với AI, ta thấy rõ ý đồ chặn đứng tận gốc rủi ro AI sinh ảo giác. Ngay khi một triệu chứng được gửi đi, hệ thống tước luôn quyền tự trả lời của LLM. Thay vào đó, nó ép Backend kích hoạt tuần tự hai tiến trình ngầm. Đầu tiên, khối Trích xuất ngữ cảnh RAG bị lôi ra quét database để bốc y văn làm điểm tựa. Chỉ khi có được bối cảnh sạch này, khối Sinh phản hồi tư vấn AI mới được cấp quyền xử lý để dán nhãn Triage và nhả chữ ra màn hình. Mất đi một trong hai mắt xích này, toàn bộ luồng tư vấn sẽ văng lỗi ngay lập tức nhằm bảo vệ an toàn y khoa.

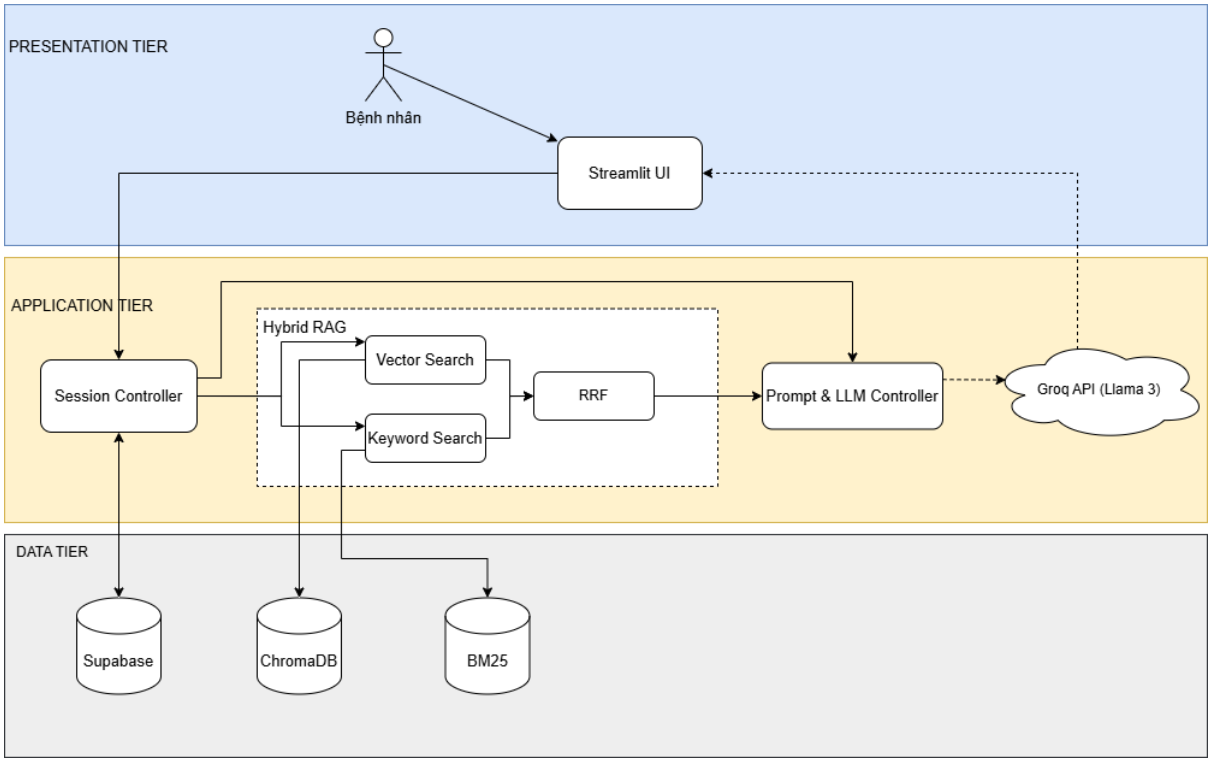
Ở một góc khác, khối Xem lịch sử tư vấn cũng bị cắm một đường <<include>> thẳng tắp vào Xác thực người dùng. Nó đóng vai trò như một cổng chặn (gatekeeper) thép. Không có chuyện chui vào xem bệnh án cũ nếu token xác thực không hợp lệ, qua đó giúp hệ thống phân tách vĩnh viễn dữ liệu tạm thời của khách vãng lai và hồ sơ bảo mật của thành viên.

Cơ chế rẽ nhánh có điều kiện (<<extend>>): Các chốt chặn phòng vệ Đời thực hiếm khi chạy mượt mà theo Happy Path. Bệnh nhân khai báo rất lung tung. Việc kiến trúc bổ sung hai luồng ngoại lệ là bước đi bắt buộc để vá các lỗ hổng logic:

- Luồng hỏi đáp bổ sung: Nhánh này mở rộng (<<extend>>) trực tiếp từ khối hỏi đáp chính. Nó chỉ nhảy ra chặn đầu khi chuỗi dữ liệu đầu vào quá mỏng, ví dụ người bệnh chỉ gõ đúng một chữ "đau bụng". Hệ thống nhận diện tham số đang rỗng, lập tức đình chỉ việc tra cứu RAG và chủ động hỏi vặn lại người dùng. Chỉ khi lấp đầy được các biến số y tế tối thiểu, tiến trình mới được thả cho đi tiếp.
- Luồng Phát cảnh báo khẩn cấp: Đây là một cú ghi đè luồng dữ liệu mang tính sinh tử. Nó cắm rễ từ khối Sinh phản hồi AI và hoạt động như một cái bẫy chực chờ. Nếu thuật toán phân tích phát hiện các từ khóa tử thần như "khó thở", "nôn ra máu", toàn bộ logic tư vấn chuyên khoa thông thường lập tức bị đạp bỏ. Nó nhường chỗ ngay cho một cảnh báo đỏ rực trên màn hình, ép người bệnh gọi cấp cứu 115 thay vì tiếp tục ngồi chat.

3.3 KIẾN TRÚC TỔNG THỂ HỆ THỐNG

Bài toán cốt lõi của ứng dụng này không nằm ở việc gọi API, mà là làm sao gánh được hai tác vụ ngốn tài nguyên khủng khiếp cùng lúc là cày xới tập y văn khổng lồ (RAG) và sinh văn bản tự nhiên (LLM Inference). Thiết kế nguyên khối bị gạt bỏ ngay từ pha phác thảo đầu tiên. Đổ dồn mọi thứ vào một server chỉ dẫn đến tình trạng cạn kiệt RAM và độ trễ không thể dung thứ. Quyết định kiến trúc cuối cùng là chẻ ngang hệ thống theo mô hình 3 tầng (3-Tier Architecture). Mục đích rất rõ ràng: tách bạch hoàn toàn lớp giao diện mỏng, phi trạng thái ở trên cùng khỏi cỗ máy nhai nuốt dữ liệu hạng nặng bên dưới.



Hình 3.3: Sơ đồ kiến trúc tổng thể hệ thống

Nhìn vào sơ đồ thiết kế, vòng đời của một luồng dữ liệu bị ép phải đi qua các chốt chặn và phân hệ riêng biệt sau:

Tầng giao diện tương tác (Presentation Tier):

Giao diện người dùng được ủy thác hoàn toàn cho Streamlit. Đây là một sự đánh đổi kiến trúc có chủ đích. Chúng ta chấp nhận vứt bỏ sự linh hoạt của các framework frontend truyền thống (như React hay Vue) để đổi lấy tính đồng nhất tuyệt đối về công nghệ: 100% Python từ đầu đến cuối. Tầng này được thiết kế cực mỏng, không chứa bất kỳ logic nghiệp vụ cốt lõi nào. Nó thuần túy là một thiết bị đầu cuối làm nhiệm vụ hứng text thô từ người bệnh và render chuỗi token do AI trả về.

Nhưng Streamlit có một tử huyệt vật lý: bản chất phi trạng thái. Cứ mỗi lần người dùng ấn Enter, nó lại kích hoạt render toàn bộ mã nguồn lại từ đầu. Để ứng dụng không bị "mất trí nhớ" sau mỗi câu hỏi, hệ thống buộc phải cắm một luồng ngầm đâm xuyên xuống database tầng đáy nhằm liên tục đồng bộ lại bối cảnh hội thoại.

Tầng xử lý logic (Application Tier):

Đây là lò phản ứng lõi của hệ thống. Tầng này lùi về sau làm nhạc trưởng, băm nhỏ request từ Streamlit thành các luồng task chạy ngầm. Ở đây có ba khối thành phần định hình nên toàn bộ sức mạnh xử lý:

– Khối Quản lý phiên (Session Controller): Chốt chặn đầu tiên. Ngay khi request rớt xuống, nó móc vào cơ sở dữ liệu để định danh người dùng, bốc toàn bộ lịch

sử trò chuyện cũ lên. Mọi thứ được nối lại thành một chuỗi văn cảnh liền mạch trước khi cấp quyền cho tiến trình đi tiếp.

- Khối Truy xuất lai (Hybrid RAG): Rủi ro lớn nhất của RAG Vector thuần túy là dễ bỏ sót từ khóa y khoa chuyên biệt. Giải pháp ở đây là bẻ đôi luồng truy vấn. Một nhánh đâm thẳng xuống không gian Vector để tìm kiếm theo nét nghĩa. Nhánh còn lại quét qua Keyword Search để bắt dính chính xác thuật ngữ. Kết quả thô từ hai luồng này lập tức bị ép qua hàm nội suy RRF (Reciprocal Rank Fusion) ngay trên bộ nhớ RAM để triệt tiêu nhiễu, chắt lọc ra những chunk tài liệu có độ tin cậy cao nhất.
- Khối Kiểm soát Prompt (Prompt & LLM Controller): Đây là màng lọc cuối cùng. Lịch sử chat và y văn sạch được gom lại, nhào nặn thành một Payload hoàn chỉnh. Quan trọng nhất, logic ngắt luồng cấp cứu được hard-code chết tại đây nhằm chặn đứng các ca nguy hiểm trước khi LLM kịp sinh chữ.

Thay vì cố chấp chạy LLM nội bộ rồi chịu trận với độ trễ phần cứng, kiến trúc thiết lập một đường ống API băng thông rộng là đường nét đứt trên sơ đồ để đẩy toàn bộ Payload sang hạ tầng Groq Cloud. Việc mượn sức mạnh chip LPU của Groq chính là lời giải duy nhất để ép Time-To-First-Token luôn nằm dưới ngưỡng 3 giây.

Tầng dữ liệu (Data Tier):

Ở tầng đáy, hệ thống tuân thủ triết lý Polyglot Persistence là lưu trữ đa mô hình. Không có chuyện nhét chung mọi thứ vào một database. Dữ liệu bị phân mảnh có chủ đích thành ba hạ tầng chuyên biệt để tối ưu hóa tốc độ truy xuất:

- Supabase (PostgreSQL): Gánh phần dữ liệu có cấu trúc. Đây là hầm chứa an toàn cho tài khoản người dùng và lịch sử các phiên chat. Nó chính là "bộ nhớ dài hạn" đặc trị căn bệnh stateless của tầng giao diện.
- ChromaDB: Kho chứa hàng triệu vector embeddings. Khối này được tinh chỉnh để tính toán khoảng cách cosine tức thì cho các truy vấn ngữ nghĩa.
- BM25 Index: Một cấu trúc chỉ mục thưa lưu trữ cục bộ. Nó lấp vào điểm mù của không gian Vector, đảm bảo hệ thống không bỏ lọt bất kỳ tên biệt dược hay con số liều lượng nào.

Sự phân lập vật lý ở tầng đáy này tạo ra một cơ chế giới hạn vùng nổ rất an toàn: giả sử tiến trình cào dữ liệu RAG bị sập, lịch sử khám bệnh của bệnh nhân nằm ở Supabase vẫn được bảo toàn nguyên vẹn.

3.4 SƠ ĐỒ KHỐI VÀ LUỒNG XỬ LÝ

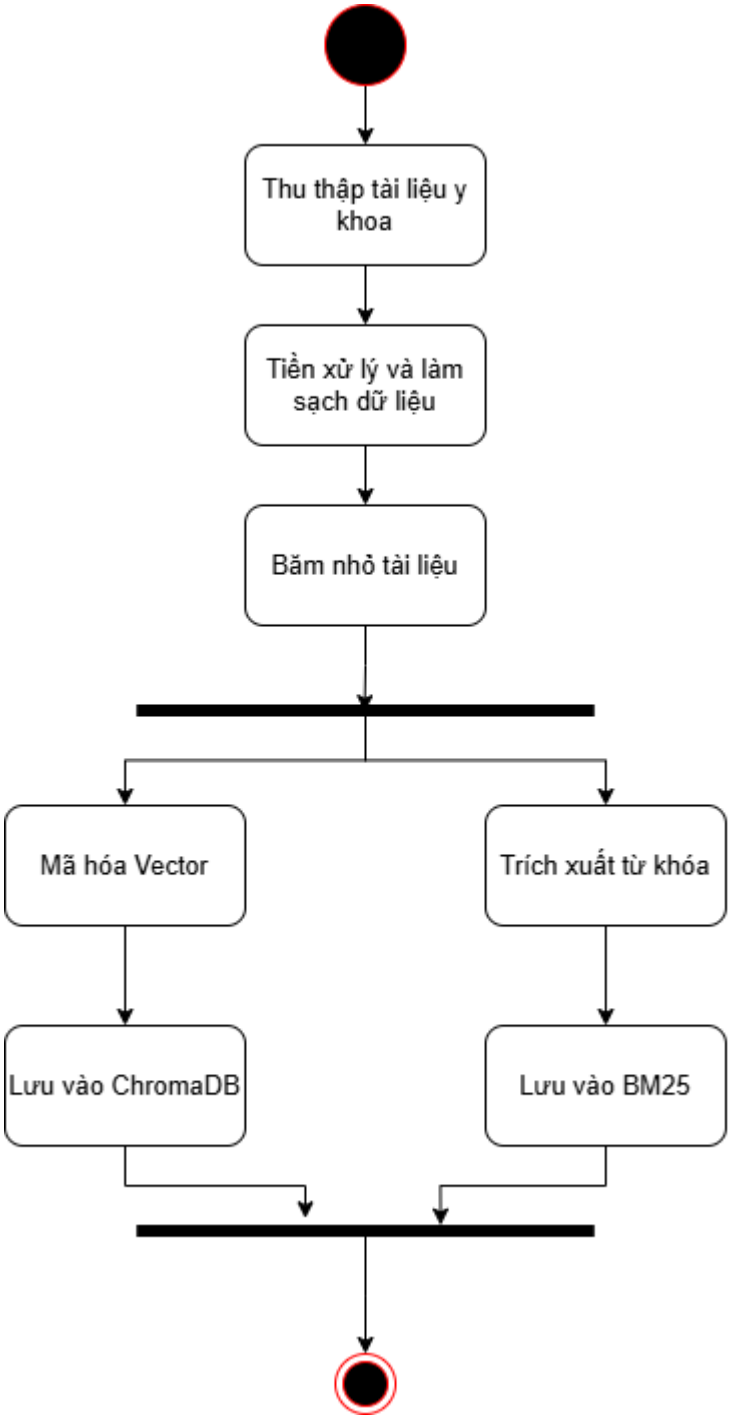
Nếu kiến trúc tổng thể là bản đồ quy hoạch các tòa nhà, thì luồng xử lý chính là hệ thống giao thông kết nối chúng. Trong một hệ thống RAG y tế, vòng đời của dữ liệu

được chia làm hai pha tách biệt hoàn toàn là pha nạp dữ liệu nền (chạy ngầm, offline) và pha truy vấn sinh văn bản (chạy thời gian thực, online).

3.4.1 Luồng xử lý dữ liệu đầu vào

Bỏ qua mớ lý thuyết hào nhoáng về AI tạo sinh, thực tế triển khai RAG trong lĩnh vực y tế luôn bị chi phối bởi một nguyên lý nghiệt ngã: "Garbage in, Garbage out". Bạn có thể đổ tiền thuê những API LLM đắt đỏ nhất, nhưng nếu nền tảng y văn bên dưới là một mớ hỗn độn, hệ thống chắc chắn sẽ sinh ảo giác. Trong y tế, một câu trả lời bịa đặt không chỉ gây lỗi phần mềm, nó là rủi ro nhân mạng.

Chính vì tính chất nghiêm ngặt này, luồng nạp dữ liệu được kiến trúc tách bạch hoàn toàn khỏi giao diện người dùng. Nó là một tiến trình chạy ngầm. Nhiệm vụ cốt lõi của phân hệ này là càn quét các kho y văn thô (PDF, văn bản), bóc tách, dọn dẹp và đúc chúng thành những cấu trúc chỉ mục tối ưu nhất, chuẩn bị sẵn sàng đạn dược cho các lượt truy vấn tốc độ cao sau này.



Hình 3.3.1: Sơ đồ hoạt động

Nhìn vào sơ đồ thiết kế, tiến trình nạp dữ liệu không chảy theo một đường thẳng cơ học mà buộc phải đi qua các trạm biến đổi và rẽ nhánh có chủ đích:

Khởi nguồn từ dữ liệu thô, hệ thống bắt đầu bằng tác vụ Thu thập và Tiền xử lý. Việc bóc tách PDF y khoa thực tế không hề màu hồng. Đây là nơi các ký tự rác, lỗi font và định dạng bảng biểu bị cày xới và làm sạch trước khi đi vào bộ nhớ tạm.

Thử thách thực sự bắt đầu ở trạm Semantic Chunking để băm nhỏ theo ngữ nghĩa. Ném nguyên một cuốn bách khoa toàn thư vào context window của LLM là điều không

tưởng vì giới hạn token. Tài liệu buộc phải băm nhỏ. Tuy nhiên, nếu dùng thuật toán cắt chuỗi cứng nhắc theo số lượng từ, chúng ta sẽ tự tay bẻ gãy ngữ cảnh, ví dụ như cắt đôi một câu định nghĩa triệu chứng. Quyết định kỹ thuật được áp dụng ở đây là Chunking có độ trễ gối đầu. Các chunk sẽ có một phần dữ liệu trùng lấp với chunk trước đó, đảm bảo tính liền mạch của y lý.

Ngay sau khi băm nhỏ, hệ thống đi vào điểm rẽ nhánh then chốt ở nút Fork trên sơ đồ. Tại đây, thay vì đẩy dữ liệu vào một DB duy nhất như các hệ thống RAG sơ sài, luồng xử lý bị chẻ làm hai tiến trình chạy song song:

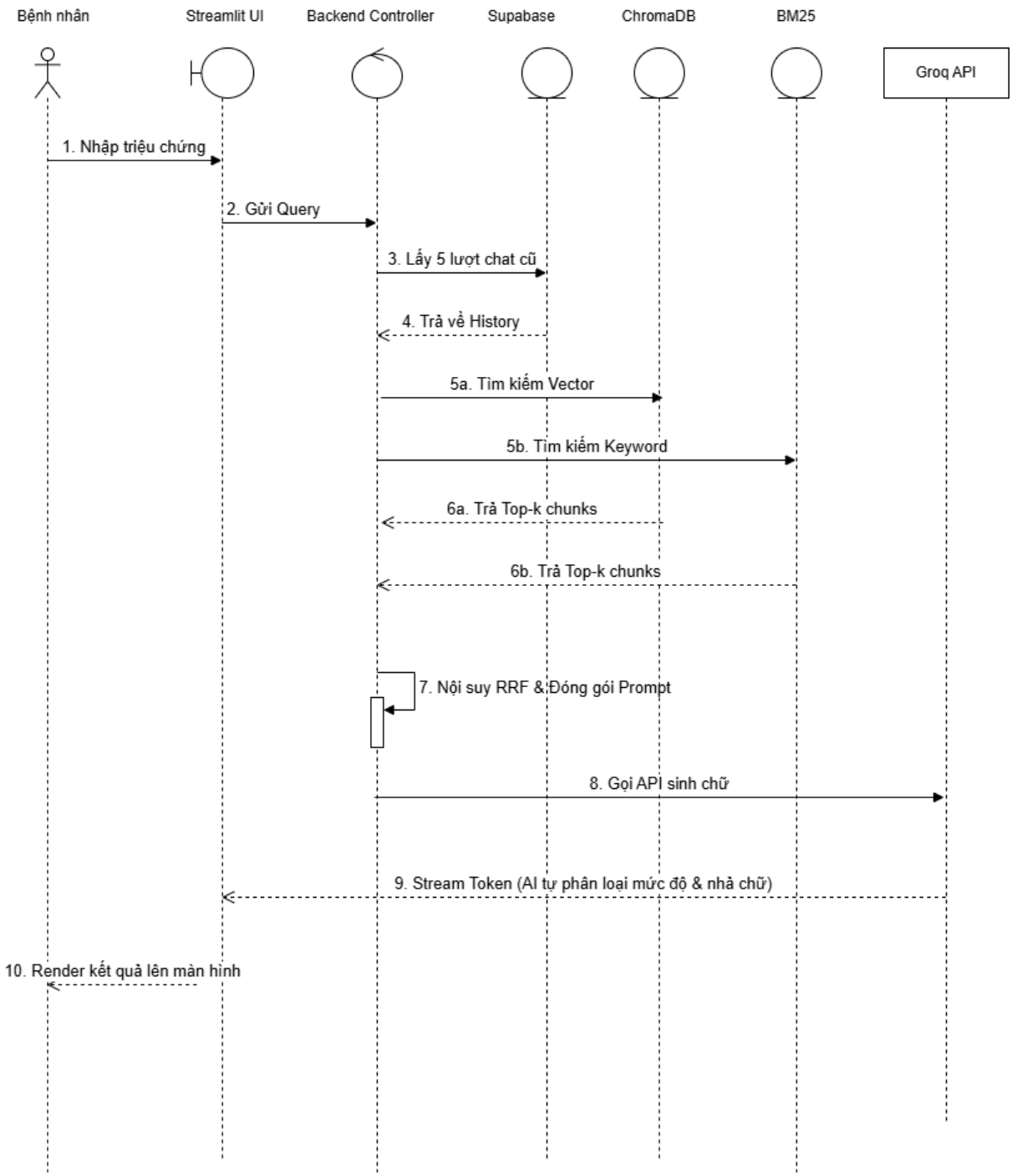
- Nhánh 1 - Mã hóa không gian Dense: Các đoạn text bị ép qua mô hình Embedding, dịch ngôn ngữ tự nhiên thành một ma trận số học hàng ngàn chiều. Những chunk có ý nghĩa tương đồng sẽ nằm co cụm lại với nhau trong không gian vector. Toàn bộ khối dữ liệu này được đổ thẳng xuống ChromaDB.
- Nhánh 2 - Trích xuất Sparse Index: Lý do tồn tại của nhánh này là vì mô hình Vector cực kỳ tệ trong việc dò tìm từ khóa chính xác. Nếu bệnh nhân hỏi về "Paracetamol 500mg" hay "Mã bệnh ICD-10", Vector thường xuyên đánh trượt. Để vá lỗ hổng này, nhánh thứ 2 dùng thuật toán TF-IDF để trích xuất tần suất các từ khóa thô và lưu gọn vào cấu trúc BM25 Index.

Cuối cùng, một chốt đồng bộ (nút Join) được đặt ở cuối đường ống. Hệ thống chỉ ghi nhận tiến trình Data Ingestion thành công và kết thúc luồng khi và chỉ khi cả hai kho ChromaDB và BM25 đều đã báo cáo nạp dữ liệu hoàn tất. Cơ chế khóa chốt này chặn đứng rủi ro bất đồng bộ, tránh tình trạng tìm thấy tài liệu ở kho Vector nhưng lại báo lỗi rỗng ở kho Keyword.

3.4.2 Luồng truy vấn và sinh văn bản

Thực tế triển khai các hệ thống AI luôn chỉ ra một rào cản chí mạng: Độ trễ. Sự khác biệt lớn nhất của luồng Online là giao tiếp thời gian thực so với luồng Offline là áp lực khổng lồ về chỉ số TTFT. Hệ thống không được phép treo hay bắt bệnh nhân nhìn màn hình trắng.

Luồng xử lý này phải giải quyết một bài toán kiến trúc cồng kềnh: Làm sao để một giao diện bản chất là phi trạng thái như Streamlit có thể nhớ mạch hội thoại, đồng thời cào dữ liệu từ hai kho DB độc lập, ép luật y tế, và gọi API xuyên lục địa,... tất cả quy trình đó phải vận hành trơn tru chỉ trong tích tắc? Khóa giải mã nằm ở thiết kế truy vấn song song và ủy quyền logic xử lý cho LLM.



Hình 3.3.2: Sơ đồ tuần tự

Nhìn vào trục thời gian của sơ đồ tuần tự (Hình 3.3.2), một vòng đời truy vấn (Query Pipeline) vận hành qua các chặng nối tiếp nhau, không có bất kỳ luồng thắt cổ chai nào:

Chặng 1: Phục dựng ngữ cảnh (Bước 1 - 4). Khi bệnh nhân nhập một câu hỏi thô cộc lốc như "Vậy tôi nên uống thuốc gì?", bản thân chuỗi string này là vô nghĩa đối với bất kỳ cỗ máy tìm kiếm nào. Giao diện Streamlit không giữ trạng thái lâu dài. Do đó, ngay khi Query được bắn xuống, Backend Controller lập tức chọc thẳng vào Supabase

để kéo 5 lượt hội thoại gần nhất lên. Chuỗi lịch sử này được ráp nối lại để tái tạo bức tranh toàn cảnh trước khi mang đi tìm y văn.

Chặng 2: Truy xuất lai song song (Bước 5a, 5b, 6a, 6b). Bối cảnh đã đủ, hệ thống kích hoạt cơ chế dò tìm Hybrid Retrieval. Điểm đáng chú ý trong thiết kế là Backend không gọi từng Database một cách rùa bò. Hai "mũi khoan" được phóng xuống tầng Data tại cùng một tọa độ thời gian: Một mũi quét qua ChromaDB (Vector Search) để gom tài liệu theo độ tương đồng ngữ nghĩa. Mũi thứ hai càn quét tệp BM25 (Keyword Search). Phải duy trì thiết kế "cổng kênh" này vì không gian Vector cực kỳ tệ trong việc bắt dính các từ khóa yêu cầu độ chính xác tuyệt đối như tên biệt dược hay mã bệnh ICD-10. BM25 ở đây hoạt động như một tấm lưới an toàn, bù đắp lỗ hổng mù từ khóa của hệ thống Vector.

Chặng 3: Nội suy RRF và Kỹ nghệ Prompt (Bước 7). Hai luồng tìm kiếm ném về Backend một danh sách tài liệu thô với cơ chế tính điểm hoàn toàn khác biệt. Lệnh tự gọi nội bộ (Self-call) ở bước 7 kích hoạt thuật toán RRF đóng vai trò trọng tài, xóc lại toàn bộ mớ bòng bong này và chốt ra những chunk tài liệu uy tín nhất.

Đồng thời, đây là nơi diễn ra quyết định thiết kế cốt lõi của toàn bộ hệ thống: Đóng gói Prompt. Khác với các hệ thống chatbot sơ xài thường viết script "If/Else" cứng ở tầng Application để chặn từ khóa nguy hiểm, kiến trúc này gói gọn toàn bộ logic y tế vào lõi Prompt. Lịch sử chat, Y văn RRF, Câu hỏi người dùng được nhào nặn cùng với một bộ luật sàng lọc khắt khe. Trọng tâm của bộ luật này là "Bẫy cấp cứu" tiêm thẳng vào ngữ cảnh của AI một chỉ thị tối cao là nếu phát hiện dấu hiệu sinh tồn nguy hiểm, lập tức chuyển trạng thái sang báo động khẩn cấp.

Chặng 4: Suy luận đám mây và Đổ dữ liệu thời gian thực (Bước 8 - 10). Khối Payload JSON hoàn chỉnh được bắn thẳng qua hạ tầng Groq API. Tại đây, mô hình Llama 3 tiếp nhận lượng context window đã được chuẩn bị sẵn. Trách nhiệm của AI lúc này không đơn thuần là "nói chuyện", mà nó trở thành một cỗ máy phân loại. Dựa trên những ràng buộc cứng trong Prompt, chính AI sẽ tự đánh giá mức độ nghiêm trọng để quyết định xem ca bệnh này cần gọi 115 hay chỉ cần theo dõi tại nhà.

Để triệt tiêu cảm giác về độ trễ mạng khi giao tiếp với server quốc tế, hệ thống không đứng đợi Llama 3 sinh xong toàn bộ văn bản rồi mới trả về một cục. Một kết nối Streaming liên tục được thiết lập. API Groq nhả ra token nào, luồng dữ liệu lập tức tuồn về Streamlit để render lên màn hình người bệnh chữ đó. Hiệu ứng gõ phím này thực chất là một thủ thuật UX sống còn, giúp đánh lừa cảm giác chờ đợi và giữ chân người dùng trong lúc cỗ máy vẫn đang miệt mài chạy suy luận ở phía sau.

3.5 THIẾT KẾ CƠ SỞ DỮ LIỆU

Trong kiến trúc của một hệ thống hỗ trợ sàng lọc y tế, cơ sở dữ liệu không đơn thuần là nơi lưu trữ các bản ghi tĩnh mà đóng vai trò là "xương sống" cho toàn bộ quy trình suy luận của LLM. Hệ thống này được xây dựng trên triết lý Hybrid RAG, một sự kết hợp giữa khả năng hiểu ngữ nghĩa sâu của Vector Database và tính chính xác tuyệt đối của Document Index truyền thống.

Việc tách biệt ba thực thể lưu trữ dưới đây không phải là sự cồng kềnh hóa hệ thống, mà là cách tôi giải quyết bài toán: Làm thế nào để AI vừa có thể "cảm" được cách diễn đạt dân dã của bệnh nhân, vừa không bị nhầm lẫn giữa các biệt dược có tên gọi gần giống nhau?

3.5.1 Cơ sở dữ liệu Vector (ChromaDB)

ChromaDB được lựa chọn làm nền tảng cho lớp Dense Retrieval (Truy xuất dày). Đây là nơi tri thức y khoa thô được chuyển hóa thành một không gian vector đa chiều, cho phép hệ thống "hiểu" thay vì chỉ "khớp" ký tự.

- Bản chất kỹ thuật: Chúng tôi sử dụng mô hình nhúng paraphrase-multilingual-MiniLM-L12-v2. Với khả năng ánh xạ văn bản y khoa vào không gian 384 chiều, mô hình này giúp hệ thống xác định được khoảng cách ngữ nghĩa giữa các triệu chứng. Một bệnh nhân than phiền về việc "đau đầu như búa bổ" sẽ được dẫn dắt tới các tài liệu về "chứng đau nửa đầu" hoặc "áp lực nội sọ" nhờ vào khoảng cách Cosine ngắn giữa các vector này, dù về mặt ký tự chúng hoàn toàn khác biệt.
- Tại sao lại là MiniLM-L12? Trong quá trình phát triển, chúng tôi ưu tiên sự cân bằng giữa độ chính xác đa ngôn ngữ đặc biệt là tiếng Việt và tài nguyên tính toán. Việc triển khai trực tiếp trên CPU mà vẫn đảm bảo tốc độ phản hồi tính bằng mili giây là yếu tố tiên quyết để duy trì chỉ số TTFT ổn định cho luồng Online.
- Giá trị thực chứng: ChromaDB đóng vai trò là bộ lọc ngữ cảnh rộng, giúp AI không bị lạc hướng khi người dùng mô tả triệu chứng bằng ngôn ngữ không định hình hoặc sử dụng các từ địa phương.

3.5.2 Cơ sở dữ liệu Document Index (BM25)

Nếu ChromaDB là một "người tư vấn" đầy cảm tính về ngôn ngữ, thì lớp Sparse Retrieval (Truy xuất thưa) dựa trên thuật toán BM25 lại là một "thư viện viên" cực kỳ nguyên tắc.

- Giải quyết lỗ hổng của Vector Search: Thực tế triển khai cho thấy các mô hình Embedding thường bị "mù" trước các danh từ riêng biệt dược hoặc các mã

bệnh ICD-10 hiếm gặp. BM25 ở đây đóng vai trò là mỏ neo chính xác. Khi một bệnh nhân nhắc đích danh "Aspirin" hay "Paracetamol", chỉ mục BM25 sẽ bắt dính các tài liệu chứa đúng các từ khóa này, thay vì gợi ý các tài liệu về "thuốc giảm đau nói chung" như cách Vector Search thường làm.

- Quy trình xử lý dữ liệu: Chỉ mục này được xây dựng sau khi văn bản qua bước tiền xử lý khắt khe bởi tokenizer underthesea. Việc lưu trữ dưới định dạng tệp .pkl giúp Backend load dữ liệu gần như tức thời vào bộ nhớ RAM, loại bỏ hoàn toàn độ trễ so với việc truy vấn qua mạng tới các Database trung tâm.
- Hội tụ RRF (Reciprocal Rank Fusion): Điểm mấu chốt nằm ở hàm _rrf_fusion. Kết quả từ ChromaDB và BM25 sẽ được trộn lại theo trọng số thứ hạng. Đây là lúc hệ thống thực hiện phép giao thoa: lấy cái thông minh của ngữ nghĩa bù cho cái chính xác của từ khóa, đảm bảo tài liệu được nạp vào Prompt luôn là tài liệu tối ưu nhất hiện có.

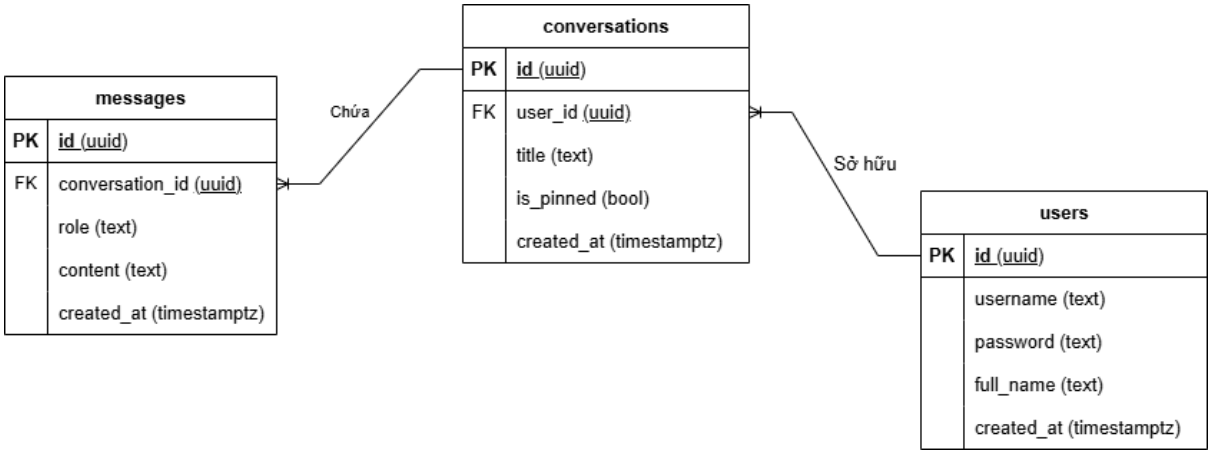
Bảng 3.1: So sánh đặc tính truy xuất trong mô hình Hybrid RAG

Đặc tính	Dense Retrieval (ChromaDB)	Sparse Retrieval (BM25)
Bản chất	Ngữ nghĩa	Từ vựng
Điểm mạnh	Hiểu ý định, từ đồng nghĩa	Chính xác tuyệt đối danh từ riêng
Độ nhạy	Cao với các đoạn văn dài	Cao với từ khóa hiếm, tên thuốc
Mục tiêu	Mở rộng không gian tìm kiếm	Khu trú phạm vi thực thể y tế

3.5.3 Cơ sở dữ liệu Quan hệ (Supabase)

Để mô hình Hybrid RAG có thể thực hiện tốt nhiệm vụ Multi-turn RAG suy luận dựa trên chuỗi hội thoại dài cho nhiều bệnh nhân cùng lúc, hệ thống không thể lưu trữ trạng thái một cách tạm bợ. Supabase được xây dựng trên lõi PostgreSQL đảm nhận vai trò quản lý định danh và duy trì ngữ cảnh dài hạn.

Thay vì nhồi nhét toàn bộ lịch sử vào một cấu trúc phẳng, kiến trúc cơ sở dữ liệu được chuẩn hóa thành ba thực thể tách biệt, liên kết chặt chẽ với nhau qua các ràng buộc khóa ngoại. Trong một môi trường phi trạng thái như Streamlit, kiến trúc này đóng vai trò là "sợi chỉ đỏ" duy nhất gắn kết các yêu cầu rời rạc của người dùng.



Hình 3.5.3: Sơ đồ thực thể quan hệ

Sự phân rã kiến trúc này giải quyết triệt để bài toán phân mảnh dữ liệu và cô lập ngữ cảnh cho từng bệnh nhân. Chi tiết đặc tả cho các thực thể như sau:

3.5.3.1 Thực thể users (Người dùng)

Tên thực thể: Bệnh nhân / Người dùng

Mô tả thực thể: Đóng vai trò gốc rễ của luồng dữ liệu, lưu trữ thông tin tài khoản và định danh cá nhân. Việc tách riêng bảng user giúp hệ thống sẵn sàng mở rộng sang mô hình phòng khám trực tuyến, nơi mỗi bệnh nhân có một không gian tư vấn hoàn toàn độc lập và bảo mật.

Chi tiết thuộc tính:

- id (Khoá chính): Mã định danh duy nhất của người dùng. Đây là điểm neo cho toàn bộ dữ liệu lịch sử khám bệnh phía sau.
- username (Text): Tên đăng nhập vào hệ thống.
- password (Text): Mật khẩu để xác thực quyền truy cập.
- full_name (Text): Họ và tên đầy đủ của bệnh nhân, dùng để cá nhân hóa giao diện UI và trích xuất vào các báo cáo y tế.
- created_at (Timestamptz): Dấu thời gian ghi nhận thời điểm khởi tạo tài khoản.

3.5.3.2 Thực thể conversations (Phiên hội thoại)

Tên thực thể: Phiên tư vấn y tế

Mô tả thực thể: Một bệnh nhân có thể hỏi về triệu chứng hô hấp vào thứ Hai, và hỏi về tiêu hóa vào tuần sau. Thực thể này sinh ra để đóng gói các chủ đề bệnh lý đó thành những phiên riêng biệt, tránh việc AI bị nhiễu ngữ cảnh từ các bệnh nền không liên quan ở quá khứ.

Chi tiết thuộc tính:

- id (Khoá chính): Mã định danh duy nhất cho mỗi phiên chat.
- user_id (Khoá ngoại): Liên kết trực tiếp về thực thể users, xác định phiên chat này thuộc sở hữu của tài khoản nào.
- title (Text): Tiêu đề tóm tắt chủ đề của phiên khám, hỗ trợ tái tạo giao diện History log trực quan cho người dùng.
- is_pinned (Bool): Cờ trạng thái đánh dấu các phiên tư vấn quan trọng.
- created_at (Timestamptz): Thời điểm bệnh nhân bắt đầu mở phiên khám mới.

3.5.3.3 Thực thể messages (Chi tiết tin nhắn)

Tên thực thể: Lượt thoại / Chi tiết tin nhắn

Mô tả thực thể: Nơi lưu trữ payload thực tế của hệ thống. Đây là kho nguyên liệu thô cuối cùng được trích xuất để ghép nối với Y văn từ kho Hybrid Retrieval trước khi đẩy vào bộ máy Prompt Controller.

Chi tiết thuộc tính:

- id (Khoá chính): Mã định danh của từng dòng tin nhắn.
- conversation_id (Khoá ngoại): Ràng buộc khắt khe ép mỗi dòng tin nhắn phải thuộc về một bối cảnh phiên khám cụ thể.
- role (Text): Phân định rạch ròi giữa lời kể của bệnh nhân và phản hồi của hệ thống. Đây là biến số sống còn để bước nhào nặn Prompt ở tầng Backend không bị nhiễu loạn logic.
- content (Text): Nội dung văn bản thô của lượt thoại.
- created_at (Timestamptz): Đảm bảo tính tuần tự tuyệt đối. Lịch sử hội thoại nếu bị đảo lộn thời gian sẽ dẫn đến việc AI hiểu sai tiến trình phát triển của bệnh lý, một sai lầm chí mạng trong chẩn đoán y tế.

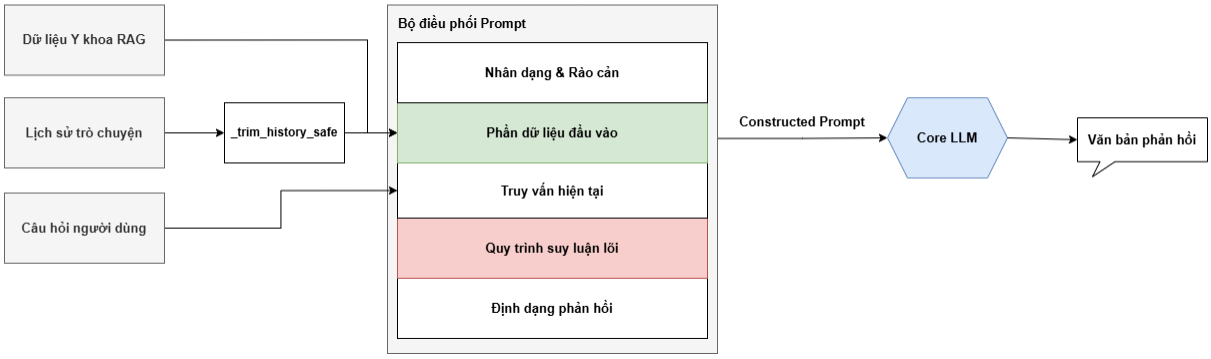
3.6 THIẾT KẾ BỘ ĐIỀU PHỐI PROMPT

Đưa một mô hình ngôn ngữ lớn (LLM) trần trụi vào môi trường y tế là một rủi ro kiến trúc cực kỳ lớn. Bản chất sinh tạo của các mô hình này luôn đi kèm với điểm mù chết người: "ảo giác y khoa". Ở góc độ thiết kế hệ thống, thay vì đốt tài nguyên vào việc tinh chỉnh trọng số mô hình là một giải pháp đắt đỏ, cứng nhắc và không thể vá lỗi realtime thì tôi chọn cách tiếp cận thực dụng hơn: xây dựng một kiến trúc Prompt Orchestrator ngay tại tầng Backend.

Mô hình lõi lúc này bị tước bỏ hoàn toàn quyền tự do sáng tạo. Nó bị nhốt trong một "sandbox" logic. Tầng Backend không đơn thuần đóng vai trò trạm trung chuyển

gọi API, mà nó hoạt động như một cỗ máy nghiền nặn, kiểm duyệt và đóng gói đa luồng dữ liệu thành một khối payload duy nhất trước khi cấp quyền cho AI xử lý.

Kiến trúc của khối payload này được tôi phân rã thành 5 lớp thông tin, xếp chồng lên nhau theo mức độ ưu tiên giảm dần.



Hình 3.6: Sơ đồ khối kiến trúc đóng gói Prompt

Lớp 1: Cấu hình nhân dạng và rào cản lõi (System Persona & Guardrails) Đây là tầng móng của toàn bộ payload. Mệnh lệnh tại đây mang tính răn đe tuyệt đối, ghi đè lên mọi hành vi mặc định của mô hình.

Thay vì một prompt mỏng manh kiểu "bạn là bác sĩ", hệ thống hard-code các rào cản phủ định gắt gao. Nó giới hạn không gian hoạt động của AI chốt ở mức "Trợ lý ảo Sàng lọc Y tế". Có những nguyên tắc tử huyệt được neo chặt ở đây: Tuyệt đối không chẩn đoán xác định. Không kê đơn thuốc dưới mọi hình thức. Thái độ giao tiếp phải giữ chừng mực, đồng cảm nhưng chuyên nghiệp. Vi phạm một trong các rào cản này, luồng suy luận bên trong sẽ bị vỡ và request tự động bị hủy.

Lớp 2: Khối dữ liệu nội suy (Data Input Section)

Thực tế triển khai RAG luôn đối mặt với rào cản về giới hạn token. Lớp này làm nhiệm vụ "vá" dữ liệu động vào prompt một cách có kiểm soát. Nó tiếp nhận hai luồng input:

Thứ nhất là ký ức hội thoại đã qua hàm tiền xử lý _trim_history_safe. Việc cắt gọt lịch sử giúp giữ lại context đồng tham chiếu để AI hiểu bệnh nhân đang nói gì, mà không làm tràn context window. Thứ hai là Grounded Context trích xuất từ engine RAG. Cụm dữ liệu này đi kèm một lệnh cấm mang tính sinh tử là "Nếu RAG trả về rỗng, tuyệt đối không được tự suy diễn hay dùng kiến thức pre-train để bịa câu trả lời". Có gì dùng nấy.

Lớp 3: Truy vấn động (User Current Query)

Đơn giản và trực diện. Đây là vùng chứa chuỗi văn bản thô vừa được parse từ request mới nhất của bệnh nhân. Nó đóng vai trò là tham số kích hoạt toàn bộ chuỗi phân tích ở tầng dưới.

Lớp 4: Khối logic suy luận (Reasoning Engine)

Đây là phân lớp phức tạp và "ăn tiền" nhất của kiến trúc. Chúng ta không bao giờ xuất đoạn này ra UI cho người dùng thấy. Nó là một bộ khung tư duy ép LLM phải suy luận theo quy trình chuẩn y khoa, biến một cỗ máy đoán chữ thành một động cơ phân tích cơ học.

Quy trình bị ép chạy qua các node logic:

- Bước 0: Bắt buộc mô hình nội suy ngược, tổng hợp History và Query hiện tại để vẽ ra bức tranh toàn cảnh lâm sàng.
- Bước 1: Pass qua bộ lọc 3 yếu tố (Triệu chứng, Thời gian, Dấu hiệu đi kèm). Nếu thiếu, trigger yêu cầu hỏi lại. Tuy nhiên, tôi thiết lập thêm một Luật phá vòng lặp tại đây. Thực tế người dùng rất hay khai "tôi không biết/không rõ". Nếu gặp case này, hệ thống phải đánh dấu pass để tránh việc bot rơi vào vòng lặp spam câu hỏi.
- Luật Red Flag (Ngắt quãng cấp cứu): Đây là một system override ghi đè hệ thống. Ngay khi bộ phân tích nhận diện các từ khóa đe dọa sinh tồn (khó thở cấp, đau ngực trái, xuất huyết...), toàn bộ bộ lọc ở Bước 1 lập tức bị ném bỏ. Luồng xử lý bị bẻ gập thẳng sang trạng thái báo động đỏ.
- Bước 2 & 3: Phân rã mức độ khẩn cấp (Đỏ, Vàng, Xanh) và chạy thủ tục Tự kiểm duyệt trước khi xuất bản. LLM phải tự soi lại output của chính nó xem có vô tình dính từ ngữ khẳng định 100% hay lọt tên thuốc nào không.

Lớp 5: Định hình khuôn mẫu đầu ra (Output Templates)

Kể cả khi LLM suy luận đúng, cách nó nhả chữ cũng cần format chặt chẽ. Lớp đáy này quy định cấu trúc trả về bằng Markdown.

Tùy thuộc vào kết quả của Lớp 4, bộ điều phối sẽ định tuyến luồng output ra 1 trong 4 hướng cơ bản là bắn cảnh báo ngoài phạm vi nếu hỏi phi y tế, chào hỏi thông thường, hỏi bù thông tin, hoặc xuất kết quả sàng lọc. Nếu lọt vào luồng sàng lọc, biểu mẫu Markdown bị ép cứng: mào đầu bằng 1-2 câu đồng cảm, tiếp đến là 3 block thông tin cô đặc gồm phân tích sơ bộ, mức độ, cảnh báo và khu vực này chỉ cho phép in 1 dòng duy nhất có icon trực quan và cuối cùng là chuyên khoa đề xuất.

CHƯƠNG 4

TRIỂN KHAI VÀ THỬ NGHIỆM HỆ THỐNG

4.1 MÔI TRƯỜNG PHÁT TRIỂN VÀ CÀI ĐẶT

4.1.1 Môi trường phần cứng

- CPU: Intel Core i5 / AMD Ryzen 5 thế hệ mới trở lên. Khâu tốn CPU nhất là thuật toán BM25 và embedding dữ liệu bằng thư viện Sentence-Transformers.
- RAM: Tối thiểu 8GB. Khi load mô hình paraphrase-multilingual-MiniLM-L12-v2 vào bộ nhớ và duy trì ChromaDB instance, hệ thống sẽ ngốn khoảng 2-3GB RAM nền.
- Ổ cứng: SSD. Bắt buộc. Tốc độ truy xuất ổ cứng quyết định trực tiếp đến độ trễ khi lôi dữ liệu từ file index .pkl và SQLite của ChromaDB.

4.1.2 Nền tảng phần mềm và thư viện

Hệ sinh thái của dự án được xây dựng hoàn toàn trên Python phiên bản 3.10+ để tận dụng tối đa các tính năng typing và match-case. Các dependencies cốt lõi được quản lý tập trung:

- Tầng giao diện: streamlit xây dựng UI nhanh gọn, native với Python.
- Tầng AI và suy luận: groq giao tiếp với Llama 3 tốc độ cao, langchain-huggingface quản lý các mô hình embedding mã nguồn mở.
- Tầng Truy xuất & Xử lý ngôn ngữ: chromadb (Vector Database), rank_bm25 (Sparse Retrieval), và đặc biệt là underthesea – bộ tokenizer chuyên dụng để cắt từ tiếng Việt, giúp BM25 hiểu đúng ngữ nghĩa thay vì chẻ nhỏ từ ghép một cách vô nghĩa.
- Tầng Dữ liệu: supabase (PostgreSQL) quản lý định danh và lịch sử hội thoại nhiều lượt.

4.2 XÂY DỰNG GIAO DIỆN NGƯỜI DÙNG

Nhằm tối ưu hóa trải nghiệm người dùng và hiện thực hóa khả năng tiếp cận hỗ trợ y tế mọi lúc mọi nơi, hệ thống đã được triển khai trực tuyến trên nền tảng đám mây. Giao diện được thiết kế theo phong cách tối giản, hiện đại, tập trung vào tính tương tác trực quan giữa người bệnh và trợ lý ảo.

Người dùng có thể truy cập và trải nghiệm hệ thống trực tiếp tại địa chỉ: https://health-chatbot-tranhuutai.streamlit.app/

Dưới đây là các thành phần giao diện chính và các tính năng tương tác cốt lõi của hệ thống:

4.2.1 Bảng điều khiển hội thoại

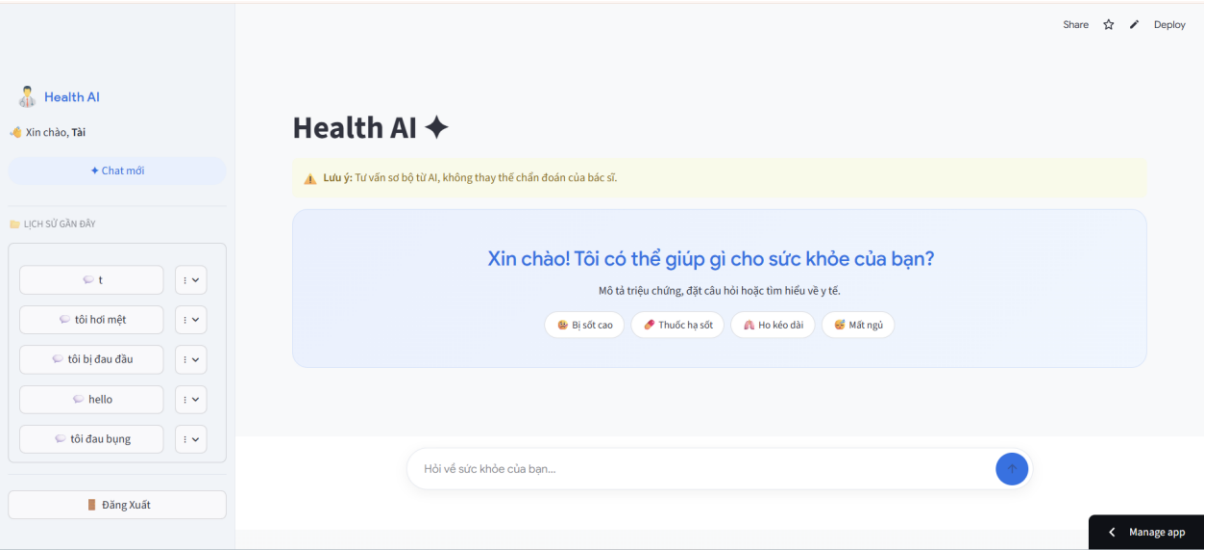
Đây là bộ não lưu trữ trí nhớ ngắn hạn của bệnh nhân. Nằm bên cột trái, Sidebar liên tục gọi API xuống Supabase để fetch danh sách các session_id. Thay vì vứt cho người dùng một màn hình trống trơn mỗi lần truy cập lại, hệ thống hiển thị lịch sử các "Phiên tư vấn" theo trình tự thời gian. Mỗi khi người dùng click vào một phiên cũ, Streamlit lập tức chọc vào bảng messages, lôi toàn bộ lịch sử đoạn chat đó nạp vào st.session_state và render lại màn hình. Cảm giác liền mạch này khiến người bệnh tin tưởng rằng AI thực sự "nhớ" bệnh án của họ.

4.2.2 Không gian tương tác chính

Khu vực trung tâm được thiết kế tối giản, loại bỏ mọi nút bấm thừa thãi để tập trung 100% vào khung chat. Điểm nhấn kỹ thuật tại đây không nằm ở màu sắc, mà nằm ở hiệu ứng dòng chảy.

Kiến trúc Hybrid RAG + Prompt 5 lớp phía sau phải xử lý một khối lượng công việc khổng lồ: quét vector, gộp BM25, chạy thuật toán RRF, đóng gói payload rồi mới ném cho Groq API. Độ trễ có thể lên tới 2-3 giây. Nếu bắt người bệnh nhìn vòng tròn xoay vô hồn trong 3 giây đó, họ sẽ thoát trang.

Giải pháp là tận dụng tham số stream=True của Groq API. Ngay khi token đầu tiên được LLM nhả ra, giao diện Streamlit lập tức in nó lên màn hình. Kỹ thuật này giảm triệt để chỉ số TTFT xuống dưới 0.5s. Người bệnh sẽ thấy AI đang "gõ" từng chữ một một cách từ tốn, tạo ra một nhịp điệu giao tiếp giống hệt như đang chat với một bác sĩ thật, đồng thời che lấp hoàn toàn thời gian delay của hệ thống mạng.



Hình 4.2: Giao diện người dùng

4.3 KỊCH BẢN THỬ NGHIỆM VÀ ĐÁNH GIÁ

4.3.1 Kịch bản 1: Hỏi đáp ngoài phạm vi y tế

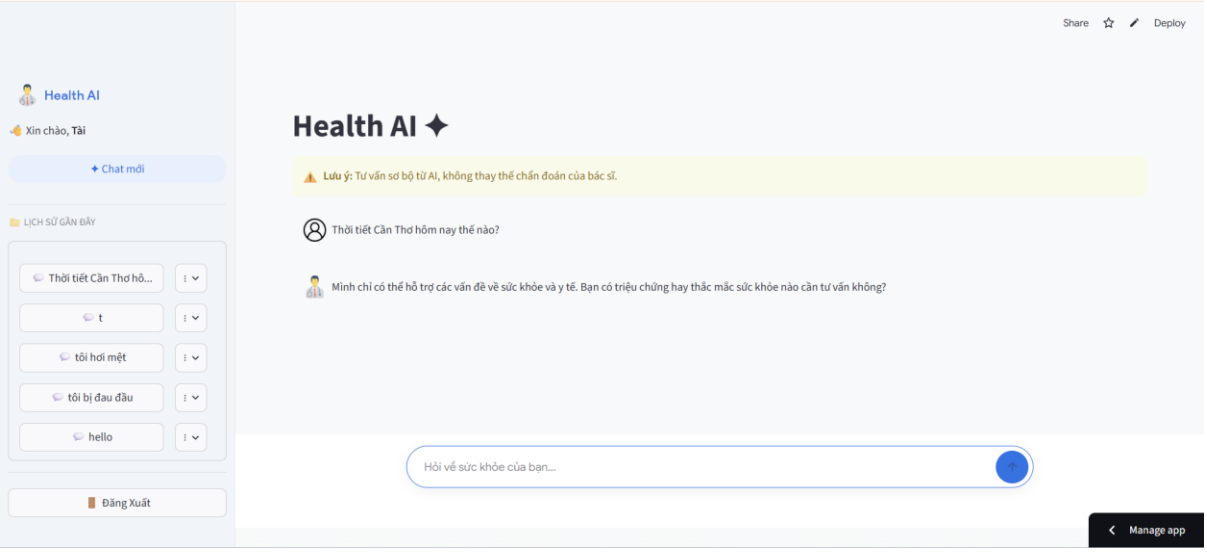
4.3.1.1 Tiêu điểm kiểm thử

Bản chất cốt lõi của mọi LLM trên thị trường đều mắc "hội chứng phục vụ" chúng luôn cố gắng làm hài lòng người dùng bằng cách đưa ra câu trả lời cho mọi câu hỏi, bất chấp ngữ cảnh. Nếu không có một lớp khiên bảo vệ, một chatbot y tế rất dễ bị "jailbreak" để trở thành công cụ làm bài tập hộ, viết code dạo hoặc tra cứu thời tiết. Bài test này cố tình nhồi một truy vấn đời sống rác để kiểm tra độ cứng của rào cản System Persona. Mục tiêu là xem hệ thống có bị đánh lừa để rời khỏi "sandbox" y tế hay không.

4.3.1.2 Dữ liệu đầu vào

"Thời tiết Cần Thơ hôm nay thế nào?"

4.3.1.3 Kết quả xử lý



Hình 4.3.1: Ảnh hỏi đáp ngoài phạm vi y tế

4.3.1.4 Đánh giá kiến trúc

Hệ thống phòng vệ hoạt động chính xác tuyệt đối ở mức thiết kế. Khi nhận truy vấn nhiễu, luồng điều khiển của khối Prompt Orchestrator đã chặn đứng yêu cầu ngay tại Lớp 5 (Hướng định tuyến 0 - Out of scope).

Điểm thành công lớn nhất ở kịch bản này không chỉ nằm ở câu từ chối lịch sự, mà nằm ở việc tối ưu tài nguyên. Bằng việc nhận diện sai lệch ý định ngay từ vòng ngoài, bộ định tuyến đã ngăn không cho hệ thống kích hoạt luồng Hybrid RAG (ChromaDB + BM25). Nghĩa là, máy chủ không lãng phí một chút tài nguyên CPU/RAM nào để lôi dữ liệu y văn ra đối chiếu với một câu hỏi về thời tiết. Mô hình LLM lõi bị ép vào thế bị động, chỉ được phép nhả ra chuỗi văn bản từ chối đã được hard-code sẵn và nhanh chóng điều hướng người bệnh quay lại chủ đề sức khỏe.

4.3.2 Kịch bản 2: Hỏi bệnh thiếu thông tin

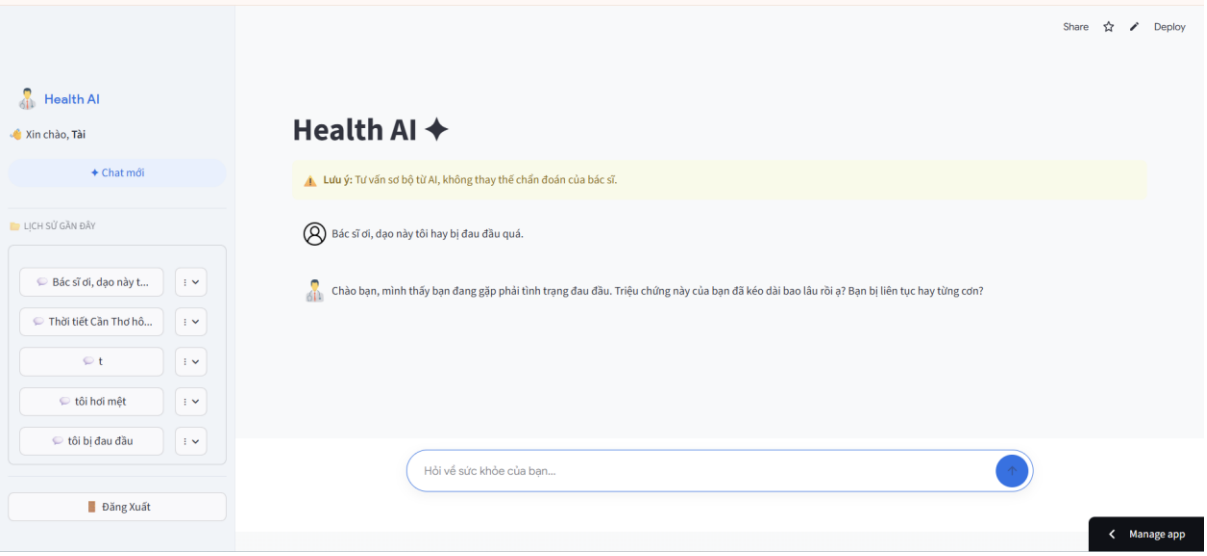
4.3.2.1 Tiêu điểm kiểm thử

Trong thực tế lâm sàng, bệnh nhân hiếm khi cung cấp một bệnh sử hoàn hảo ngay từ câu chào hỏi. Tiêu điểm của kịch bản này không nằm ở việc AI trả lời hay như thế nào, mà nằm ở năng lực "kìm hãm" của hệ thống. Thay vì để LLM trượt dài vào ảo giác y khoa dựa trên một triệu chứng đơn độc, hệ thống phải biết cách từ chối đưa ra kết luận, đồng thời kích hoạt cơ chế Slot-filling để thu thập đủ cấu trúc dữ liệu trước khi cho phép luồng Hybrid RAG hoạt động hết công suất.

4.3.2.2 Dữ liệu đầu vào

"Bác sĩ ơi, dạo này tôi hay bị đau đầu quá". Input: Chỉ có duy nhất 1 yếu tố là Triệu chứng. Thiếu Thời gian và Dấu hiệu đi kèm.

4.3.2.3 Kết quả xử lý



Hình 4.3.2: Ảnh hỏi bệnh thiếu thông tin

4.3.2.4 Đánh giá kiến trúc

Nhìn từ bề mặt giao diện, đây chỉ là một lượt tương tác ngắn gọn. Nhưng dưới góc độ thiết kế hệ thống, phản hồi này minh chứng cho sự thành công của kiến trúc phân tầng Guardrails.

Khi nhận chuỗi truy vấn thô "tôi hay bị đau đầu", các LLM thông thường rất dễ bị "sập bẫy" từ khóa. Trọng số của từ "đau đầu" đủ lớn để kích hoạt cơ chế sinh văn bản tự do, lôi toàn bộ dữ liệu về bệnh lý thần kinh ra và nôn ra một bảng chẩn đoán hoành tráng. Trong lĩnh vực y tế, việc máy vội vàng khuyên đi khám chỉ dựa trên một câu than vãn là một lỗi sai logic cực kỳ nguy hiểm.

Ở đây, Bộ điều phối Prompt đã can thiệp mượt mà. Nó chạy ngầm bộ lọc 3 yếu tố và nhanh chóng phát hiện payload đầu vào mới chỉ đạt 1/3 điều kiện. Ngay lập tức, luồng định tuyến bị bẻ gập. Hệ thống khóa chặt chức năng phân tích sâu, tước bỏ hoàn toàn quyền sử dụng biểu mẫu chẩn đoán Markdown của AI.

Mô hình lúc này bị ép lùi lại, đóng vai trò của một nhân viên điều phối mẫn cán. Câu phản hồi được sinh ra thỏa mãn cùng lúc hai tiêu chí khắt khe: thứ nhất là thái độ đồng cảm để làm dịu tâm lý người bệnh, thứ hai là tính định hướng thu thập dữ liệu ("kéo dài bao lâu", "từng cơn hay liên tục").

Toàn bộ quá trình đánh giá rủi ro, đếm yếu tố và chuyển hướng luồng dữ liệu này được xử lý hoàn toàn trong "hộp đen" của backend. Người dùng không phải nhìn thấy bất kỳ chuỗi logic máy móc nào trên màn hình. Đồng thời, cơ chế Stateful lưu trữ trạng thái hội thoại cũng được hệ thống hâm nóng, chuẩn bị sẵn sàng ghép nối câu trả lời tiếp theo của bệnh nhân vào bộ nhớ ngắn hạn để tạo thành một Query hoàn chỉnh cho lượt truy xuất RAG sắp tới.

4.3.3 Kịch bản 3: Hỏi bệnh đủ thông tin

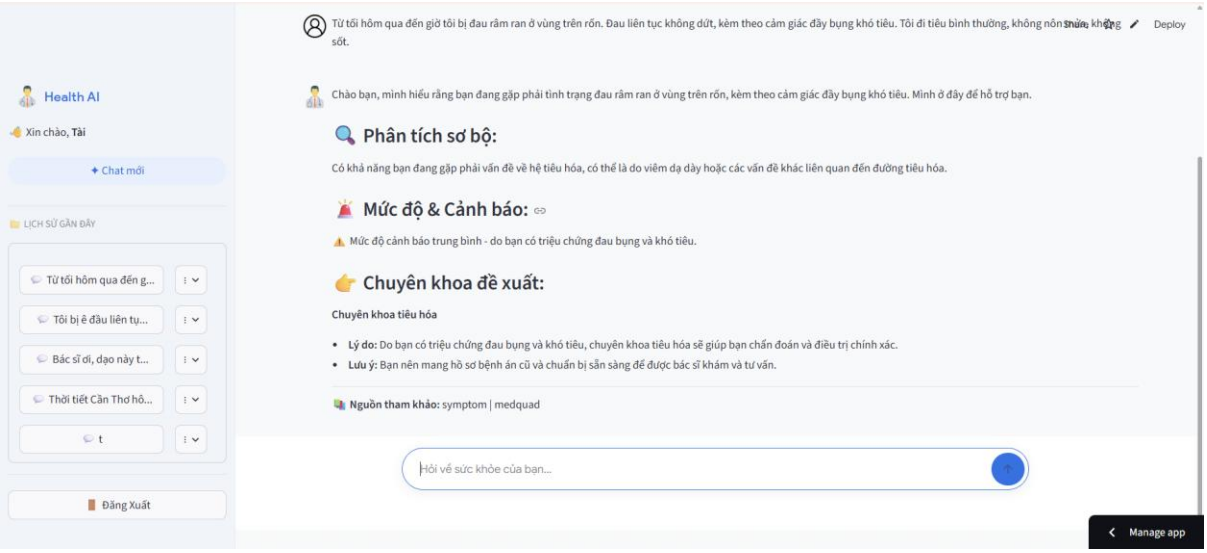
4.3.3.1 Tiêu điểm kiểm thử

Đây là bài test hiệu năng cốt lõi của toàn bộ kiến trúc. Khi người bệnh đã cung cấp một bệnh sử đủ độ chín, hệ thống phải làm được ba việc cùng lúc: vượt qua rào cản đếm yếu tố, truy xuất chính xác dữ liệu từ kho RAG, và ép mô hình ngôn ngữ lớn (LLM) nhả chữ theo đúng khuôn mẫu lâm sàng mà không lấn sân sang việc chẩn đoán bệnh.

4.3.3.2 Dữ liệu đầu vào

"Từ tối hôm qua đến giờ tôi bị đau râm ran ở vùng trên rốn. Đau liên tục không dứt, kèm theo cảm giác đầy bụng khó tiêu. Tôi đi tiêu bình thường, không nôn mửa, không sốt". Input: Khối dữ liệu thỏa mãn hoàn toàn bộ lọc logic. Triệu chứng: đau râm ran vùng trên rốn. Thời gian: từ tối hôm qua đến giờ. Dấu hiệu kèm tính chất như đau liên tục, đầy bụng khó tiêu, không nôn mửa, không sốt.

4.3.3.3 Kết quả xử lý



Hình 4.3.3: Ảnh hỏi bệnh đủ thông tin

4.3.3.4 Đánh giá kiến trúc

Luồng thực thi minh chứng sự vận hành hoàn hảo của kiến trúc Hybrid RAG kết hợp Prompt Orchestrator trong nhánh xử lý chính.

- Vượt rào cản logic: Bộ suy luận ngầm nhận diện chính xác dữ liệu đầu vào đạt ngưỡng 3/3 yếu tố và an toàn nhờ các từ khóa phủ định "không nôn", "không sốt", từ đó kích hoạt trực tiếp Hướng định tuyến 3.
- Tuân thủ biểu mẫu: LLM bị ép khuôn thành công. Phản hồi không bị lan man thành văn xuôi mà tuân thủ tuyệt đối cấu trúc Markdown được thiết kế. Các khối thông tin (Phân tích sơ bộ, Mức độ cảnh báo, Chuyên khoa đề xuất) được render tách biệt, sạch sẽ.
- An toàn y khoa và RAG: Hệ thống truy xuất đúng ngữ cảnh thể hiện qua phần nguồn tham khảo. Rào cản Guardrails hoạt động hiệu quả khi AI kìm chế được thói quen chẩn đoán chắc chắn, sử dụng chuẩn xác các cụm từ giảm nhẹ rủi ro pháp lý ("Có khả năng", "Có thể là do").

4.3.4 Kịch bản 4: Tình huống Cấp cứu

4.3.4.1 Tiêu điểm kiểm thử

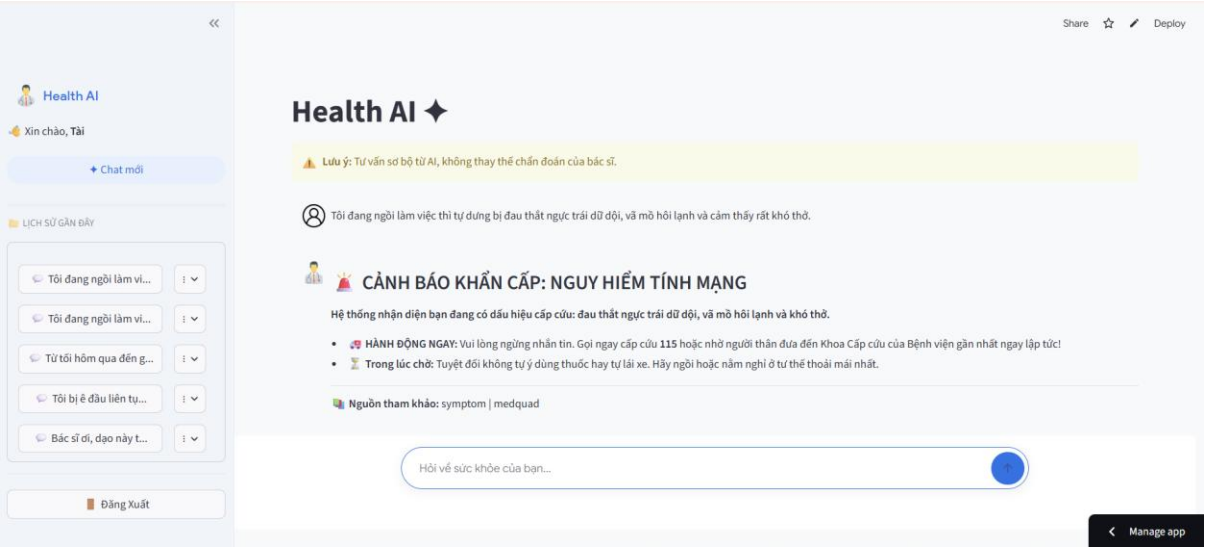
Kiểm thử rào cản "Ngoại lệ Cấp cứu". Trọng tâm đánh giá khả năng nhận diện rủi ro sinh tử của hệ thống: Khi phát hiện tín hiệu nguy hiểm, khối logic có khả năng "đạp phanh" khẩn cấp, chém đứt luồng gạn hỏi thông thường để bẻ ngoặt control flow sang giao thức báo động đỏ hay không? Trong thực tế lâm sàng, tính mạng con người không cho phép độ trễ, và chắc chắn không có chỗ cho việc để AI ngồi dông dài thu thập bệnh sử.

4.3.4.2 Dữ liệu đầu vào

"Tôi đang ngồi làm việc thì tự dưng bị đau thắt ngực trái dữ dội, vã mồ hôi lạnh và cảm thấy rất khó thở". Dưới lăng kính của Rule Engine, khối văn bản này không phải là một câu giao tiếp thông thường. Nó chứa cụm semantic tokens mang trọng số rủi ro kịch trần của chứng nhồi máu cơ tim: đau thắt ngực trái dữ dội, vã mồ hôi lạnh, khó thở.

Đây là một phép thử cực hạn nhắm vào tính năng ngắt mạch. Sự xuất hiện của chùm từ khóa Red Flag này buộc backend phải tuân thủ luật ngoại lệ một cách tuyệt đối: Đình chỉ ngay lập tức logic kiểm tra tính đầy đủ của dữ liệu, bypass mọi rule đếm yếu tố và ném trả một phản hồi cấp cứu khẩn cấp ngay tại lượt hội thoại đó.

4.3.4.3 Kết quả xử lý



Hình 4.3.3: Ảnh tình huống cấp cứu

4.3.4.4 Đánh giá kiến trúc

Luồng thực thi chứng minh hệ thống đạt chuẩn an toàn của một ứng dụng y tế (Medical-grade Triage System).

- Cơ chế ghi đè: Ngay tại Lớp 1 (Suy luận ngầm), hệ thống nhận diện từ khóa nguy hiểm và lập tức kích hoạt ngoại lệ cấp cứu. Nó tự động đánh dấu tổng = 3/3, bỏ qua hoàn toàn hướng định tuyến 2 (Hỏi bù thông tin).
- Chốt chặn an toàn: Hệ thống không cố gắng chẩn đoán rườm rà hay an ủi dài dòng. Khối lệnh Hướng 3 được thực thi với Cảnh báo mức độ cao nhất (Khẩn cấp/Cấp cứu ngay), đồng thời ưu tiên đề xuất tuyến đầu là Khoa Cấp cứu hoặc gọi xe cứu thương (115), khóa chặt rủi ro chậm trễ y tế.

4.4 ĐÁNH GIÁ HIỆU NĂNG VÀ ĐỘ CHÍNH XÁC

Thước đo thực tế của hệ thống này không nằm ở các bộ số liệu benchmark bề nổi, mà ở khả năng kiểm soát luồng thực thi và tính kỷ luật của LLM trong một môi trường rủi ro cao như y tế.

Bài toán hóc búa nhất về mặt trải nghiệm là việc tích hợp cơ chế suy luận ngầm ngay trong luồng Streaming. Nếu để lộ chuỗi logic tính toán đếm yếu tố, trải nghiệm người bệnh sẽ sụp đổ. Giải pháp được chứng minh qua thực tế là thuật toán Stream Interceptor tại tầng backend: nó hoạt động như một bộ đệm "nín thở" hứng và nuốt trọn vẹn toàn bộ đoạn text rác nằm giữa cặp thẻ <thinking> do Groq API trả về, sau đó mới bắt đầu xả phần văn bản giao tiếp thực sự ra ngoài. Kết hợp với việc can thiệp trực tiếp mã CSS ở tầng Front-end để triệt tiêu lỗi hiển thị icon anchor link mặc định của Streamlit, hệ thống mang lại một luồng phản hồi liền mạch, sắc nét và sạch sẽ như một sản phẩm thương mại.

Tuy nhiên, sự mượt mà của giao diện chỉ là phần vỏ; giá trị cốt lõi bộc lộ qua độ chính xác trong định tuyến và an toàn lâm sàng. Qua các đợt stress-test bằng nhiều kịch bản khác nhau, kiến trúc Prompt Orchestrator đã chứng minh được khả năng "cầm cương" Llama 3. Thay vì để mô hình trượt dài vào ảo giác chẩn đoán bừa bãi khi đối mặt với những câu hỏi ngắn, bộ đếm yếu tố ép nó phải lùi lại, kích hoạt cơ chế Slot-filling để gạn hỏi đúng một trọng tâm còn thiếu. Khắc nghiệt hơn, hệ thống đã vượt qua bài test sinh tử về rào cản Red Flag. Khi va chạm với các chùm từ khóa nguy hiểm như "đau thắt ngực" hay "khó thở", bộ định tuyến lập tức chém đứt luồng gạn hỏi thông thường, ghi đè toàn bộ quy trình để tung ra cảnh báo gọi 115. Tính kỷ luật sắt đá này của bộ quy tắc đã bù đắp hoàn hảo cho bản chất phi tuyến tính của các mô hình sinh tạo.

Mảnh ghép cuối cùng làm nên độ chính xác của hệ thống là khả năng ép khuôn đầu ra. Một LLM luôn có xu hướng "ngứa nghề" thích viết văn xuôi lê thê. Khi luồng Hybrid RAG (kết hợp BM25 và ChromaDB) truy xuất thành công ngữ cảnh và bơm vào Prompt ở Hướng định tuyến số 3, mô hình bị khóa chặt bởi các lệnh cấm. Nó buộc phải tuân thủ mệnh lệnh in ra chính xác ba khối tiêu đề ### định sẵn, không được gộp ý tự do, và bắt buộc dùng các cụm từ giảm nhẹ rủi ro pháp lý (như "Có khả năng", "Có thể là"). Việc tước bỏ sự ngẫu hứng, ép LLM chấp hành định dạng đầu ra một cách máy móc chính là chốt chặn dứt điểm, biến hệ thống từ một chatbot trò chuyện thông thường thành một cỗ máy phân loại y tế có cấu trúc và đáng tin cậy.

CHƯƠNG 5

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 KẾT QUẢ ĐẠT ĐƯỢC

Trải qua quá trình nghiên cứu và thực nghiệm, đề tài đã hoàn thành mục tiêu xây dựng một hệ thống "Health AI" hỗ trợ sàng lọc y tế thông minh. Những kết quả cụ thể bao gồm:

- Xây dựng thành công kiến trúc Hybrid RAG: Hệ thống không chỉ dựa vào tri thức tĩnh của LLM mà đã tích hợp hiệu quả kho dữ liệu y khoa (ChromaDB) thông qua cơ chế truy xuất ngữ cảnh. Việc kết hợp tìm kiếm ngữ nghĩa và từ khóa (BM25) giúp nội dung tư vấn có độ tin cậy cao và bám sát dữ liệu chuyên môn.
- Thiết kế bộ điều phối Prompt Orchestrator: Đây là thành tựu cốt lõi về mặt logic. Hệ thống đã hiện thực hóa được cơ chế "suy luận trước khi phản hồi", giúp kiểm soát chặt chẽ luồng hội thoại, định tuyến chính xác người dùng theo các cấp độ thông tin từ chào hỏi, gạn hỏi bù dữ liệu đến chẩn đoán sơ bộ và báo động cấp cứu.
- Xây dựng thành công phân hệ quản lý người dùng: Thay vì chỉ là một giao diện chat ẩn danh, hệ thống đã được tích hợp module xác thực (Đăng ký/Đăng nhập) thông qua Supabase và quản lý phiên bản tin nhắn. Người dùng có thể lưu trữ, ghim các cuộc hội thoại quan trọng hoặc sử dụng hệ thống dưới dạng chế độ khách một cách linh hoạt.
- Tối ưu Trải nghiệm Giao diện (UI/UX) trên nền Streamlit: Đã can thiệp thành công vào kiến trúc CSS mặc định để xử lý mượt mà độ trễ sinh thông qua các hiệu ứng tải thời gian thực. Bổ sung các rào cản kỹ thuật như cơ chế chống dịch tự động giúp giữ nguyên vẹn cấu trúc hiển thị y khoa.
- Triển khai thực tế trên môi trường Cloud: Dự án đã vượt qua giai đoạn chạy thử nghiệm cục bộ để hiện diện trực tuyến trên nền tảng Streamlit Cloud, sẵn sàng phục vụ người dùng từ xa với trải nghiệm mượt mà.
- Đảm bảo an toàn lâm sàng: Hệ thống đã chứng minh được tính kỷ luật thông qua các luật ngoại lệ, ưu tiên tuyệt đối tính mạng người dùng trong các tình huống khẩn cấp thay vì chỉ tập trung vào việc xử lý ngôn ngữ thuần túy.

5.2 CÁC HẠN CHẾ CÒN TỒN TẠI

Dù đạt được những bước tiến về mặt kiến trúc, hệ thống vẫn tồn tại những rào cản kỹ thuật cần nhìn nhận thẳng thắn:

- Sự phụ thuộc vào API bên thứ ba: Việc sử dụng Groq Cloud giúp tối ưu tốc độ nhưng đồng thời tạo ra sự phụ thuộc vào hạ tầng và giới hạn băng của nhà cung cấp. Trong các tình huống đứt gãy kết nối quốc tế, độ trễ hệ thống có thể bị ảnh hưởng nghiêm trọng.
- Giới hạn về ngữ cảnh đa lượt: Dù đã áp dụng cơ chế cắt tỉa lịch sử hội thoại, nhưng khi cuộc trò chuyện kéo dài quá mức, khả năng duy trì sự mạch lạc và ghi nhớ các chi tiết nhỏ trong bệnh sử vẫn có thể gặp sai số cục bộ.
- Dữ liệu chuyên sâu còn mỏng: Kho dữ liệu RAG hiện tại chủ yếu tập trung vào các nhóm bệnh phổ biến. Đối với các ca bệnh hiếm hoặc cần sự kết hợp đa chuyên khoa phức tạp, hệ thống đôi khi vẫn phải dựa nhiều vào kiến thức tổng quát của mô hình nền (Llama 3).

5.3 HƯỚNG PHÁT TRIỂN TRONG TƯƠNG LAI

Để nâng tầm "Health AI" từ một dự án học thuật thành một công cụ hỗ trợ y tế thực chiến, các hướng phát triển tiếp theo sẽ tập trung vào:

- Xây dựng Đồ thị tri thức y khoa (Medical Knowledge Graph - MKG): Vượt qua giới hạn của không gian Vector (vốn chỉ so khớp ngữ nghĩa), hướng đi cốt lõi là cấu trúc hóa dữ liệu y tế thành một mạng lưới Đồ thị. Các thực thể y khoa (Triệu chứng, Bệnh lý, Thuốc, Chống chỉ định) sẽ được định danh thành các Node, liên kết với nhau bằng các Cạnh (Edges) thể hiện mối quan hệ logic nghiêm ngặt (ví dụ: có_triệu_chứng, gây_ra_bởi, điều_trị_bằng). Điều này giúp hệ thống lưu trữ tri thức y học dưới dạng mạng lưới logic có thể nội suy được.
- Chuyên sâu hóa vào chẩn đoán Viêm phổi ở trẻ em: Thay vì sàng lọc tổng quát, mô hình sẽ được thu hẹp phạm vi để giải quyết một bài toán y khoa phức tạp và có tính rủi ro cao: Viêm phổi nhi khoa. Tri thức đưa vào MKG sẽ bám sát các phác đồ điều trị chuẩn (như IMCI của WHO), tập trung vào việc mô hình hóa các chỉ số sinh tồn đặc thù theo độ tuổi của trẻ (nhịp thở theo tháng tuổi, dấu hiệu rút lõm lồng ngực, tím tái) nhằm hỗ trợ chẩn đoán chính xác và kịp thời.
- Nâng cấp kiến trúc lên GraphRAG (Knowledge Graph-based RAG): Tích hợp Đồ thị tri thức (MKG) với sức mạnh tạo sinh của LLM. Thay vì chỉ truy xuất các đoạn văn bản thô từ ChromaDB, AI sẽ thực hiện các truy vấn truy vết trên đồ thị (Graph Traversal). Kiến trúc GraphRAG cho phép hệ thống thực hiện suy luận đa bước (Multi-hop reasoning) để tìm ra các mối liên hệ ẩn giữa nhiều triệu chứng phức tạp của bệnh nhi, từ đó sinh ra kết luận chẩn đoán với độ chính xác lâm sàng vượt trội.

- Tăng cường tính minh bạch và khả năng giải thích (Explainable AI - XAI): Trong y tế, bác sĩ không thể tin tưởng một "hộp đen" AI chỉ đưa ra kết quả. Bằng việc ứng dụng MKG, hệ thống trong tương lai có khả năng "giải thích" tại sao nó đưa ra nghi ngờ viêm phổi. Kết quả trả về sẽ kèm theo luồng suy luận trực quan (trích xuất trực tiếp đường đi từ các Node triệu chứng đến Node bệnh lý trên đồ thị), giúp hệ thống đóng vai trò như một trợ lý hỗ trợ quyết định Lâm sàng (CDSS) thực thụ cho các bác sĩ nhi khoa.

TÀI LIỆU THAM KHẢO

[1] Lewis, P., et al. (2020), "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks", *Advances in Neural Information Processing Systems (NeurIPS)*.

[2] LangChain Documentation (2026), *Building applications with LLMs through composability*.